SurveyHub: Project Deliverable 1
Parts A–F

CPE 334 Software Engineering
Semester 1/2026

67070501040 Worawut Sereethai and 67070501059 Chanon Lhumsa-ard

DRAFT – release tagged; remaining evidence gaps listed

Evidence snapshot: 13 November 2026
Theme 10: Review, Feedback, and Survey System
Context: university course feedback and survey collection
Repository: SurveyHub
Screenshot commit: `b0e58eb9fbf0` (PR #16)
Staging verification: `00494b26df89`
Release tag: `v0.1.0-sprint1` -> `827ed220e6d2`

Contents

Project Deliverable 1 - PDF Navigation 5

1 Part A: Software Requirements Specification (SRS) and Product Scope 6
1. Stakeholder Context and Problem Statement . . . 6
1.1 Chosen context . . . 6
1.2 Product purpose . . . 6
1.3 Product name and deployment target . . . 6
2. Actors and Responsibilities . . . 7
2.1 Role model decision . . . 7
3. Product Scope . . . 7
3.1 In scope for the whole product . . . 7
3.2 Sprint 1 scope (Deliverable 1) . . . 8
3.3 Explicit exclusions . . . 8
3.4 Scope changes from the previous baseline . . . 9
3.5 Requirement summary . . . 9
4. Functional Requirements . . . 10
5. Business Rules . . . 11
5.1 Access-to-raw-data policy . . . 12
6. Non-Functional Requirements . . . 13
7. Assumptions and Architecture Decisions . . . 14
7.1 Assumptions . . . 14
7.2 Architecture decisions . . . 14

ADR-01 - Stack . . . 14
ADR-02 - Authentication and revocation . . . 15
ADR-03 - Duplicate-submission strategy . . . 15
ADR-04 - Seeded accounts only . . . 15
ADR-05 - Forward-compatible data model . . . 15
ADR-06 - Theme interpretation . . . 16
ADR-07 - Raw-data policy . . . 16
ADR-08 - Time handling . . . 16
ADR-09 - Test database . . . 16
ADR-10 - Final video length . . . 16
ADR-11 - Docker Compose in Sprint 1 . . . 16
8. Questions and Resolutions . . . 16
9. Traceability and Status . . . 17

2 Part B: Feature Inventory **19**
1. Feature Inventory . . . 19
1.1 Feature allocation summary . . . 19
1.2 Question type inventory . . . 20
1.3 F-06 acceptance criteria (Sprint 2) . . . 20
Part B (continued): Requirements Traceability . . . 22
2. Traceability Matrix . . . 22
3. Coverage and Gap Analysis . . . 24

3 Part C: System-Level Software Design Specification (SDS) **27**
1. Architecture and Component Responsibilities . . . 27
1.1 Architecture style . . . 27
1.2 Stack decision . . . 27
1.3 Layer responsibilities . . . 27
1.4 Business-rule enforcement map . . . 28
2. Data Model . . . 29
2.1 Entity relationships . . . 29
2.2 Design decisions . . . 29
2.3 Key entities, fields and constraints . . . 30
2.4 Constraints added by SQL migration . . . 30
2.5 Migration and seed strategy . . . 31
2.6 Test database . . . 31
3. UI Structure and Behaviour . . . 32
3.1 Routes . . . 32
3.2 Shared UI states . . . 32
3.3 Accessibility and usability conventions . . . 32
4. API Design and Conventions . . . 33
4.1 Envelope . . . 33
4.2 Status codes . . . 33
4.3 Error codes . . . 34
4.4 Endpoint table . . . 34
4.5 Validation approach . . . 35
5. Authentication and Authorization . . . 36
5.1 Password handling . . . 36

5.2 Token lifecycle . . . 36
5.3 Role and ownership matrix . . . 36
5.4 Authorization rules . . . 37
6. Configuration and Delivery Design . . . 37
6.1 Environment variables . . . 37
6.2 Local reproduction . . . 38
6.3 Docker Compose local environment (Sprint 1) . . . 38
6.4 CI/CD and GCP delivery plan (Lab 5) . . . 39
7. Anonymous-Mode Privacy Design (Sprint 2) . . . 39
7.1 Separation . . . 39
7.2 What the owner sees . . . 39
7.3 Residual risk . . . 40
8. Forward-Compatibility Checklist . . . 40
9. Traceability . . . 40

4 Part D: Three-Sprint Product Development Plan 41
1. Product Vision and Sprint Allocation . . . 41
2. Sprint Matrix . . . 41
3. Sprint 1 Scope . . . 41
3.1 In scope . . . 41
3.2 Out of scope for Sprint 1 . . . 42
3.3 Sprint 1 completion criteria . . . 42
4. Dependencies and Critical Path . . . 42
4.1 Feature dependencies . . . 42
4.2 Critical path for Sprint 1 . . . 43
4.3 Sprint 1 issue map . . . 43

5 Part E: Sprint 1 Feature Contracts and Software Test Specifications 44
1. Product Definition of Done (DoD) . . . 44
Part E - F-01 Authentication, Session and RBAC . . . 45
2. Contract F-01: Authentication, Session and RBAC . . . 45
Part E - F-02 Survey Lifecycle and Authoring . . . 49
3. Contract F-02: Survey Lifecycle and Authoring . . . 49
Part E - F-03 Survey Taking, Submission and Response Rules . . . 54
4. Contract F-03: Survey Taking, Submission and Response Rules . . . 54
Part E - F-04 Aggregate Summary . . . 59
5. Contract F-04: Aggregate Summary . . . 59
Part E (continued): Shared System Contracts and Test Specifications . . . 63
6. Health Endpoint Reference . . . 63
7. Supporting Unit Tests . . . 63
8. Security Matrix Tests . . . 63
9. End-to-End Tests . . . 64
10. Test Count Summary . . . 64
11. TDD Protocol for BR-01 . . . 65
12. Contract Version History . . . 65

6 Part F: Sprint 1 Verification and Engineering Evidence **67**
1. Version and reproduction summary 67
2. Build and test results recorded by feature PRs 67
3. Acceptance criteria to test results 68
4. Demonstration scenarios 69
5. Actual test excerpts 73
6. TDD evidence for BR-01 73
7. Selected issue, PR, review and merge chains 74
8. Contribution record 74
9. Sprint 1 evidence gaps, failures and disclosures 75

Project Deliverable 1 - PDF Navigation

Page ranges are generated from the Part A-F labels and update when the document is compiled twice.

Part	Source document(s)	Page range	Relevant IDs and contents
A - SRS and product scope	SRS	6–18	FR-01..FR-16, BR-01..BR-11, NFRs, assumptions and decisions
B - Feature inventory and traceability	Feature inventory; traceability	19–26	F-01..F-08; FR/BR/NFR-to-feature, sprint and verification mappings
C - System-Level SDS	System design	27–40	Architecture, data model, UI/API, authentication, authorization and delivery design
D - Sprint plan	Sprint plan	41–43	Sprint 1-3 goals, feature IDs, dependencies, outcomes and issue map
E - Sprint 1 contracts and STS	Part E overview; F-01; F-02; F-03; F-04; shared STS	44–66	F-01..F-04; AC-F01..F04; STS-F01..F04, STS-U, STS-SEC and STS-E2E
F - Verification and engineering evidence	Verification and engineering evidence	67–76	PR-reported checks, test mapping, demo evidence gaps, release status, contributions and limitations

1 Part A: Software Requirements Specification (SRS) and Product Scope

Status note: this section defines requirements and design intent. Feature and test status is summarized in Parts B and F; Part F distinguishes pull-request reports from release-level verification.

---

1. Stakeholder Context and Problem Statement

1.1 Chosen context

The product is used by a university course team. A **course coordinator** (Survey Owner) needs to collect structured feedback from **enrolled students** (Respondents) during and after a course. The current process uses ad hoc forms, shared spreadsheets and chat messages. It has four problems:

1. **No lifecycle control.** A form cannot be reliably opened, closed or versioned, so responses arrive after the feedback window and results mix different question sets.
2. **Duplicate and invalid responses.** The same student can submit several times, and required questions are often skipped, which distorts totals.
3. **Weak privacy boundaries.** Raw answers are copied into shared files. Students can see each other's comments, and there is no clear rule for who may read raw data.
4. **Manual aggregation.** The coordinator counts responses by hand and cannot see a trustworthy summary while the survey is open.

1.2 Product purpose

SurveyHub lets an organization define surveys, control their lifecycle, collect validated responses from eligible users, and report aggregate results. The system enforces its rules on the server, not in the browser.

The theme title also mentions **Review and Feedback**. In this product that requirement is expressed through **open-text feedback questions** attached to a survey. There is no separate review or rating entity outside a survey. This interpretation is recorded as a scope decision (ADR-06) and should be confirmed with the instructor or TA.

1.3 Product name and deployment target

- Working name: **SurveyHub**
- Deployment target from Lab 5 onward: Docker image on Google Cloud Run with a managed PostgreSQL instance.
- Sprint 1 runs locally with a reproducible seed and does not require cloud resources.

---

2. Actors and Responsibilities

Actor	Description	Responsibilities and permissions
Survey Owner	Course coordinator, lecturer or administrator who runs a survey	Create, edit and delete **draft** surveys; publish and close **own** surveys; view aggregate summaries for **own** surveys; no access to another owner's surveys
Respondent	Enrolled student or other eligible user	Browse published surveys; submit one validated response per survey; view only their own submission receipt; cannot manage surveys or read another respondent's answers
Unauthenticated user	Anonymous visitor without a valid token	Can call login and health endpoints only; every protected endpoint returns `401`

2.1 Role model decision

Roles are **exclusive**: an account is either `SURVEY_OWNER` or `RESPONDENT`. An owner account cannot submit a response, even to another owner's survey. This keeps the authorization matrix small and testable for Sprint 1. If a coordinator also needs to answer a survey, they use a respondent account.

This decision replaces the contradictory statements in the previous draft, where owners were allowed to open a survey form but forbidden to submit it.

---

3. Product Scope

3.1 In scope for the whole product

1. Authentication with login, logout, current-user identity and server-side token revocation.
2. Two roles with backend-enforced permissions and per-record ownership.
3. Survey lifecycle: `DRAFT -> PUBLISHED -> CLOSED`. `CLOSED` is terminal.
4. Survey authoring: title, description, ordered questions, required flags and type-specific options.
5. Question types: single-choice and short text in Sprint 1; multi-choice, long text, yes/no, five-point Likert, 1-5 rating, bounded integer and date questions in Sprint 2.
6. Response collection with validation, duplicate prevention and concurrency protection.
7. Identified responses in Sprint 1; anonymous responses with identity/answer separation in Sprint 2.
8. Response eligibility: all authenticated respondents in Sprint 1; an explicit allowlist in Sprint 2.

9. Aggregate summary: response counts, per-option counts and percentages, rating/integer statistics, date counts and unlinked short- and long-text answers.
10. Rich analysis, individual response browsing and CSV/JSON export in Sprint 3.
11. Docker Compose local development from Sprint 1; CI/CD and GCP deployment from Lab 5 onward.

3.2 Sprint 1 scope (Deliverable 1)

- React single-page client, Express/TypeScript API and PostgreSQL through Prisma.
- Seeded accounts only; no self-registration.
- Owner can create a draft survey with questions, update a draft, delete a draft, publish it and close it.
- Respondent can browse published surveys, open the form, submit one valid response and see a receipt.
- Owner can view an aggregate summary of their own survey.
- Business rules BR-01 to BR-09 and the Sprint 1 aggregate portion of BR-11 enforced in the service layer, with database constraints where useful.
- Automated verification of rules, persistence and access control, plus selected end-to-end journeys.
- Docker Compose local stack with the application and PostgreSQL, database readiness gating, a named persistent database volume, and a separate test database (GitHub Issue #2).

3.3 Explicit exclusions

Excluded item	**Reason / planned handling**
Self-registration, email verification, password reset	Seeded accounts are sufficient for the course; excluded from all sprints
Real email or SMS invitations	Not required; eligibility is represented in the database
Payments, file upload, matrix/grid questions, skip logic	Outside the theme brief; not planned
Multi-tenancy, organization administration, SSO	Outside scope
Real-time dashboards or websockets	Not required; summaries are computed on request
Native mobile application	Web only
CI/CD automation and GCP production deployment	Lab 5; Sprint 1 provides a reproducible local environment only.
Production-grade rate limiting and WAF	Recorded as technical debt; basic input limits only

Continued on next page

Excluded item	Reason / planned handling
Automated scheduled jobs	Sprint 2 auto-close is evaluated lazily on read/write, not by a background worker

3.4 Scope changes from the previous baseline

The previous draft promised editing and deleting surveys in the UI but defined no API for it. It also used a schema that could not represent multi-select answers, ratings or anonymous responses without a breaking migration. This baseline records the following approved changes:

Change	Reason
Added `GET /api/surveys/:id`, `PUT /api/surveys/:id`, `DELETE /api/surveys/:id`	Make draft authoring real and make BR-07 testable
Added `AnswerOption` join table and `Answer.numberValue`	Support multi-choice and rating without redesigning answers later
Expanded F-06 from multi-choice and rating to a defined nine-type catalogue	Give the product useful answer formats while keeping Sprint 1 limited to single-choice and short text; record the additional Sprint 2 scope before implementation (GitHub Issues #1 and #9).
Added `SurveyEligibility` and `ResponseClaim` models	Make eligibility and anonymous duplicate prevention explicit and forward-compatible
Added `User.tokenVersion` and server-side logout	Logout must invalidate the token, not just clear browser state
Added `GET /api/health`	Required for container and cloud health checks
Replaced the unmeasured performance claim with a defined benchmark	The previous ”24 tests in 2.65 s” figure was not a latency measurement
Added a Docker Compose application and PostgreSQL stack to Sprint 1	Team decision: reproducible local setup from the first scaffold; CI/CD and GCP remain Lab 5 work
Keep authored deliverable documents in the tracked `docs/` tree	Team workflow decision; only local reference material under `docs/materials/` is ignored

3.5 Requirement summary

Group	Total	Sprint 1	Later
Functional requirements	16	10	6
Business rules	11	10 (including the Sprint 1 portions of BR-02, BR-08 and BR-11)	1
Non-functional requirements	7	6	1
Total	**34**	**26**	**8**

The Sprint 1 count in this table is the same count used by the Part B coverage matrix. BR-02, BR-08 and BR-11 have a Sprint 1 portion and a later extension; they are counted

once in the Sprint 1 column.

---

4. Functional Requirements

Stable IDs are used in Parts B-F. "Sprint" is the planned delivery sprint.

ID	Requirement	Observable behavior	Sprint
FR-01	Authentication and logout	A user can log in with email and password, receives a JWT, and can log out. Logout revokes existing tokens for that account.	1
FR-02	Current user and role	`GET /api/auth/me` returns the authenticated user's id, email, full name and role, and never returns the password hash.	1
FR-03	Manage survey drafts	A Survey Owner can list and read their surveys, create a survey with a title, optional description and ordered questions, and delete a survey while it is `DRAFT`. A new survey is always `DRAFT`.	1
FR-04	Author and edit draft questions	A Survey Owner can add, edit, reorder and remove questions and options while the survey is `DRAFT`.	1
FR-05	Publish survey	A Survey Owner can publish their own valid draft. The status becomes `PUBLISHED` and `publishedAt` is recorded.	1
FR-06	Close survey	A Survey Owner can close their own published survey. The status becomes `CLOSED` and `closedAt` is recorded.	1
FR-07	Browse published surveys	A Respondent can list published surveys and see whether they have already submitted.	1
FR-08	Take and submit survey	A Respondent can open a published survey, answer the questions and submit exactly one response.	1
FR-09	Persisted confirmation	A successful submission is stored in PostgreSQL and the respondent receives a receipt with response id and timestamp. The receipt can be retrieved later.	1
FR-10	Aggregate summary	A Survey Owner can view total responses and per-question answered counts. Choice questions show option counts and percentages; short- and long-text answers appear unlinked; rating and integer questions show count, average, minimum and maximum; date questions show counts by calendar date. Sprint 1 delivers counts, choice summaries and short text; F-06 adds the remaining formats in Sprint 2.	1 (base) / 2 (F-06 formats)
FR-11	Anonymous responses	A survey can run in anonymous mode. Responses are stored without a link to the respondent, while duplicate submission is still prevented.	2

Continued on next page

ID	Requirement	Observable behavior	Sprint
FR-12	Expanded question types	In addition to Sprint 1 single-choice and short text, owners can author and respondents can answer multi-choice, long text, yes/no, five-point Likert, 1-5 rating, bounded integer and date questions. Results preserve the answer type and are included in the owner summary.	2
FR-13	Export results	A Survey Owner can export their own survey results as CSV and JSON. Anonymous surveys export aggregates only.	3
FR-14	Eligibility management	A Survey Owner can restrict a survey to an explicit set of eligible respondents. An empty list means all respondents are eligible.	2
FR-15	Scheduled close	A Survey Owner can set a closing date and time. After that time the survey behaves as closed for submissions.	2
FR-16	Browse individual responses	A Survey Owner can browse individual identified responses for their own survey. Anonymous surveys never expose identity.	3

5. Business Rules

ID	Rule	Rejection and observable behavior	Sprint
BR-01	One response per respondent per survey	A second submission by the same respondent returns `409 DUPLICATE_SUBMISSION`. Exactly one response is stored, including under concurrent requests. Enforced by a service check, a `ResponseClaim` unique constraint and an identified-response unique constraint.	1
BR-02	Survey state guard	Responses are accepted only while the survey is `PUBLISHED` and, from Sprint 2, before `closesAt`. A `DRAFT` or `CLOSED` survey returns `400 SURVEY_NOT_ACCEPTING_RESPONSES`.	1 (state) / 2 (scheduled close)
BR-03	Answer validation	Every required question must be answered and each answer must use exactly the value shape for its type. Selected options must belong to that question. Short text is trimmed and at most 2,000 characters; long text is trimmed and at most 10,000 characters. Yes/no has exactly one fixed `Yes` or `No` option; Likert has exactly one of five ordered labels; rating is an integer from 1 through 5; integer answers respect any configured minimum and maximum; date answers are calendar dates without a time zone. Invalid input returns `400 INVALID_ANSWER` or `400 MISSING_REQUIRED_QUESTIONS`. Sprint 1 enforces the single-choice and short-text subset.	1 (subset) / 2 (extensions)

Continued on next page

ID	Rule	Rejection and observable behavior	Sprint
BR-04	Survey ownership boundary	Only the owner may read, update, delete, publish, close or summarize a survey. A non-owner receives `403 FORBIDDEN_OWNERSHIP`; an unknown survey id receives `404 NOT_FOUND`.	1
BR-05	Cross-response privacy	A respondent may retrieve only their own response receipt. Requesting another respondent's receipt returns `403 FORBIDDEN_CROSS_RESPONSE_ACCESS` without returning any answer data.	1
BR-06	Publish structure	A survey can be published only if it has at least one question and every prompt is non-empty. Single-choice and multi-choice questions have at least two options; multi-choice selection bounds are valid and do not exceed the option count. Yes/no has the fixed `Yes` and `No` options, Likert has five ordered labels, rating uses the fixed 1-5 scale, and configured integer bounds do not contradict each other. Otherwise `400 INVALID_QUESTION_STRUCTURE`. Sprint 1 enforces the single-choice subset.	1 (subset) / 2 (extensions)
BR-07	Immutable structure after publication	Questions and options cannot be added, edited, reordered or removed after publication. `PUT /api/surveys/:id` returns `400 SURVEY_NOT_EDITABLE`. Metadata edits are also blocked in Sprint 1 for simplicity.	1
BR-08	Response eligibility	Only authenticated users with the `RESPONDENT` role may submit. From Sprint 2, if a survey has eligibility entries, the respondent must be on the list; otherwise `403 NOT_ELIGIBLE`. In Sprint 1 an empty eligibility list means all respondents are eligible.	1 (role) / 2 (allowlist)
BR-09	Terminal closed state	`CLOSED` is terminal. Publishing a published or closed survey, or closing a draft or closed survey, returns `400 INVALID_STATE_TRANSITION`.	1
BR-10	Anonymous identity separation	In anonymous mode, `SurveyResponse.respondentId` is `NULL`; no name, email, IP address or user agent is stored with the response; the `ResponseClaim` row records only that the user submitted, not which response is theirs. Owners see aggregates and unlinked text only.	2
BR-11	Result privacy	Aggregate and export responses never include respondent identity fields. Identified raw-response browsing is available only to the owning Survey Owner and only for identified surveys.	1 (aggregate) / 3 (raw)

5.1 Access-to-raw-data policy

The theme brief requires an explicit raw-data policy.

Caller	Aggregate summary	Individual response content	Respondent identity
Respondent	No	Own receipt only	Own account

Continued on next page

Caller	Aggregate summary	Individual response content	Respondent identity
Survey Owner (own survey, identified mode)	Yes	From Sprint 3, through an explicit response browser and export	From Sprint 3, included only in the identified raw export
Survey Owner (own survey, anonymous mode)	Yes	Unlinked text answers in the summary; no response-level identity	Never
Survey Owner (another owner's survey)	No	No	No
Unauthenticated	No	No	No

Sprint 1 implements the aggregate column only. Short-text answers appear in the summary as a list of strings with no respondent name, email or response id. This is deliberate: the previous draft claimed owners could see only aggregates while also returning raw text samples, which was contradictory.

---

6. Non-Functional Requirements

Each NFR has a measurable target and a named verification method. Targets are compared with actual results only in Part F, after execution.

ID	Quality attribute	Measurable target	Verification method	Sprint
NFR-SEC-01	Protected API enforcement	100% of protected endpoints return `401` with no private data when called without a token or with an invalid token	Parameterized integration test that enumerates every protected route (`server/tests/security-matrix.test.ts`)	1
NFR-SEC-02	Role and ownership enforcement	100% of forbidden role/ownership operations return `403` and leave the database unchanged	Role/ownership matrix integration tests across all protected operations	1
NFR-SEC-03	Password storage	Every stored password hash uses bcrypt cost 10 (`$2b$10$` prefix). No plaintext password appears in the database or API responses.	Integration test queries `users.password_hash` and asserts the prefix; response-shape assertions	1
NFR-SEC-04	Token revocation	A token used before logout returns `401 TOKEN_REVOKED` after logout	Integration test: login, call `/me`, logout, call `/me` with the same token	1
NFR-PERF-01	Read latency	p95 latency of `GET /api/surveys/:id/form` is below 300 ms over 50 sequential requests on the local development environment (Node.js 20+ and PostgreSQL 16+ on the same machine; actual versions are recorded in Part F)	`server/scripts/benchmark-read.ts` records p50/p95/p99 and writes a JSON result	1

Continued on next page

ID	Quality attribute	Measurable target	Verification method	Sprint
NFR-DATA-01	Relational integrity and durability	Foreign keys and unique constraints exist; a submitted response is still retrievable after an API restart and after a database server restart	Migration inspection plus a documented restart check against the seeded environment	1
NFR-PRIV-01	Anonymous unlinkability	In anonymous mode, `survey_responses.respondent_id IS NULL` for 100% of anonymous responses, and no identity field exists in the answers or API output	Database audit test and API response-shape assertions	2

Explicit non-target: the project does not claim production-scale throughput, high availability or sub-second p99 latency. Those are outside the course scope.

---

7. Assumptions and Architecture Decisions

7.1 Assumptions

The following assumptions are treated as true for planning. If one changes, the affected requirements, contracts and tests are revised and the change is recorded.

ID	Assumption	Impact if false
A-01	Test commands ran on one Windows workstation using Node and a loopback PostgreSQL 18.1 test database. A clean-clone run and the second student's environment remain unverified.	
A-02	Sprint 1 uses seeded accounts; no self-registration or email delivery is required.	Authentication scope and the seed/login flow must be redesigned.
A-03	One account has exactly one role; a person who needs both roles uses two accounts.	The role matrix and authorization tests must be extended.
A-04	A survey is owned by one owner account; ownership transfer is not required.	The ownership model and BR-04 change.
A-05	Timestamps are handled in UTC on the server and displayed in local time by the client.	Scheduled close and receipt timestamps become ambiguous.
A-06	Demo data is synthetic; no real student data or production secrets are used.	The raw-data policy and submission may violate privacy rules.
A-07	PostgreSQL supports the constraints and transactional behaviour used by Prisma.	The duplicate guard and data-integrity claims need a different mechanism.

7.2 Architecture decisions

ADR-01 - Stack

React 18 + Vite + TypeScript + Tailwind CSS on the client; Node.js 20+ + Express + TypeScript on the server; Prisma ORM with PostgreSQL 16+. The team already used

this stack in Labs 1-4. A modular monolith is sufficient and keeps deployment to one API container plus one database.

ADR-02 - Authentication and revocation

JWT in the `Authorization: Bearer` header, signed with HS256, containing `sub`, `role` and `ver`. `User.tokenVersion` is compared on every authenticated request. Logout increments the version, which revokes every token issued to that account. Tokens expire after 8 hours. There are no refresh tokens; the user logs in again after expiry. This is a deliberate, testable session design rather than a client-only logout that leaves the token valid.

ADR-03 - Duplicate-submission strategy

Defense in depth:

1. The service checks for an existing `ResponseClaim` and returns a friendly `409`.
2. A unique constraint on `ResponseClaim(surveyId, userId)` prevents a race.
3. `SurveyResponse(surveyId, respondentId)` is also unique for identified responses.
4. Prisma error `P2002` is translated to `409 DUPLICATE_SUBMISSION`.

ADR-04 - Seeded accounts only

Sprint 1 uses four seeded accounts: two owners and two respondents. Self-registration and password recovery are excluded. Demo passwords are synthetic and documented only in the private handoff or local seed.

ADR-05 - Forward-compatible data model

The Sprint 1 schema already contains the structures needed by later sprints:

- `AnswerOption` join table, so one answer can reference several options for multi-choice and one ordered option for yes/no or Likert.
- `Answer.numberValue` for 1-5 ratings and bounded integer questions; `Answer.dateValue` for dates.
- `Question.numberMin/numberMax` for optional integer bounds; short and long text share `textValue` with type-specific length limits.
- `SurveyEligibility`, so eligibility can be switched on without a new table.
- Nullable `SurveyResponse.respondentId` plus `ResponseClaim`, so anonymous mode does not require a breaking migration.

`schema.prisma` is the single source of truth; migrations are committed.

ADR-06 - Theme interpretation

The survey's short-text questions are the "review and feedback" element. A separate review entity is not built. This avoids inventing a second domain that the stakeholder brief does not require.

ADR-07 - Raw-data policy

Defined in section 5.1. The rule is: aggregate data is available to the owner; individual response content is limited to the owner's own survey; respondent identity is never exposed to other respondents, and is never exposed for anonymous surveys.

ADR-08 - Time handling

All timestamps are stored in UTC (`timestamptz` semantics through Prisma `DateTime`). The API returns ISO 8601 strings. The client displays local time. Scheduled close compares `closesAt` with the server clock.

ADR-09 - Test database

Integration tests use a separate PostgreSQL database (`surveyhub_test`) and run with `jest --runInBand`. Each suite truncates tables in dependency order before seeding fixtures. Tests never touch the development or demo database. This is required for reproducible evidence.

ADR-10 - Final video length

The course handouts disagree: the group specification and Final Delivery section 2 allow up to 10 minutes, while Final Delivery section 3.5 says a maximum of 4 minutes. The team will target **4 minutes** (the stricter limit) unless the instructor or TA confirms otherwise.

ADR-11 - Docker Compose in Sprint 1

Decision: Sprint 1 Issue #2 provides a root-level Compose setup for the application and PostgreSQL. The database has a health check, a named persistent volume, and separate development and test databases. The application waits for database readiness. CI/CD and GCP deployment remain Lab 5 work.

Reason: the team asked for a reproducible, containerized local environment as part of the first scaffold. This supersedes the planning draft's Lab 5 timing for Docker.

Verification: a clean-clone setup, health check, migration, seed, test-database isolation and persistence check are recorded in Part F after they are run.

8. Questions and Resolutions

#	Question	Resolution
Q1	How do we prevent two simultaneous submissions from the same respondent?	Service pre-check plus `ResponseClaim` unique constraint plus identified-response unique constraint. Catch `P2002` and return `409`. Tested with concurrent requests.
Q2	Who may read raw answers?	The owner of the survey, for their own identified survey, from Sprint 3. Respondents see only their own receipt. Anonymous surveys never expose identity. Sprint 1 exposes aggregates and unlinked text only.
Q3	What happens if an owner edits questions after publication?	Rejected with `400 SURVEY_NOT_EDITABLE`. Metadata edits are also blocked in Sprint 1 to keep the published contract stable.
Q4	How is logout made real with stateless JWT?	`User.tokenVersion` is embedded in the token and checked on every request. Logout increments it. Old tokens return `401 TOKEN_REVOKED`.
Q5	Who is eligible in Sprint 1?	Every authenticated `RESPONDENT`. The `SurveyEligibility` table is present but empty, which means "open to all respondents". Sprint 2 adds allowlist management.
Q6	Does anonymous mode need cryptography?	No. It needs separation: the response row has no user link, the claim row has no response link, and no request metadata is stored. Residual risk: a database administrator could correlate timestamps. That risk is documented rather than overclaimed.
Q7	How do we model the expanded question types without redesigning later?	`AnswerOption` stores single-choice, multi-choice, yes/no and Likert selections; `Answer.textValue` stores short and long text; `Answer.numberValue` stores ratings and bounded integers; `Answer.dateValue` stores calendar dates. `Question.minSelections/maxSelections` and `numberMin/numberMax` hold type-specific bounds.
Q8	What is the evidence standard for this project?	Link every result to a real command and commit. Identify pull-request-reported results separately; do not claim release verification before testing the release commit.
Q9	The handouts give different video limits. Which do we follow?	4 minutes unless the TA confirms 10.
Q10	Student ids end in 9 and 0; why Theme 10?	Theme 10 matches the partner's last digit (0). Theme 9 was also available but Survey was the team's preference. Recorded here as the theme decision.

9. Traceability and Status

- Every FR, BR and NFR in this document is mapped to a feature, sprint, acceptance criterion and verification target in **Part B**.

- Feature contracts for Sprint 1 are in **Part E**.
- The verification plan and evidence-collection procedure are in **Part F**.
- No requirement in this SRS is marked complete until the corresponding test has run against the submitted commit and the result is recorded in Part F.

2 Part B: Feature Inventory

Status note: this feature inventory is the scope baseline. The traceability matrix records PR-reported implementation and test status; Part F distinguishes those reports from release-level verification.

---

1. Feature Inventory

Feature	Name	Short description	Linked requirements	Depends on	Sprint
F-01	Authentication and RBAC	Login, logout with token revocation, current-user identity, JWT verification, bcrypt password storage, role guards	FR-01, FR-02; NFR-SEC-01..04	None	**1**
F-02	Survey lifecycle and authoring	List, create, read, update and delete draft surveys; ordered questions and options; publish and close; ownership, structure and immutability rules	FR-03..FR-06; BR-04, BR-06, BR-07, BR-09	F-01	**1**
F-03	Response submission and rules	Browse published surveys, load a form, submit one validated response, retrieve the receipt, reject duplicates and invalid states/answers, protect cross-response access	FR-07..FR-09; BR-01, BR-02, BR-03, BR-05, BR-08	F-01, F-02	**1**
F-04	Aggregate summary	Owner-only summary with total responses, answered counts, option counts/percentages and unlinked short text. F-06 defines additional type-aware summary values in Sprint 2.	FR-10 (Sprint 1 base; Sprint 2 type extensions); BR-04, BR-05, BR-11	F-01..F-03	**1**
F-05	Anonymous mode	Survey-level anonymous mode with identity/answer separation and duplicate prevention through claims	FR-11; BR-10; NFR-PRIV-01	F-01, F-03, F-04	**2**
F-06 (Issue #9)	Expanded question types	Multi-choice, long text, yes/no, five-point Likert, 1-5 rating, bounded integer and date questions across authoring, response and summary	FR-12	F-02, F-03, F-04	**2**
F-07	Eligibility and scheduled close	Explicit respondent allowlist and a closing date/time evaluated by the server	FR-14, FR-15; BR-02, BR-08	F-02, F-03	**2**
F-08	Rich analytics, response browsing and export	Charts, identified raw-response browser and CSV/JSON export with mode-aware privacy	FR-13, FR-16; BR-11	F-04, F-06	**3**

1.1 Feature allocation summary

Sprint	Features	Checkpoint
Sprint 1	F-01, F-02, F-03, F-04	Deliverable 1 - 8 November 2026
Sprint 2	F-05, F-06, F-07	Deliverable 2 - 29 November 2026; Labs 5-6
Sprint 3	F-08 plus hardening and defect fixes	Final Delivery - 13 December 2026; Lab 7

Every feature is allocated to exactly one sprint. Later work is not used to excuse a missing Sprint 1 feature.

1.2 Question type inventory

Question type	Sprint	Answer representation and rule
Single-choice	1	Select exactly one question option.
Short text	1	Trimmed text, up to 2,000 characters.
Multi-choice	2	Select question options within the configured minimum and maximum.
Long text	2	Trimmed text, up to 10,000 characters.
Yes/no	2	Select exactly one fixed option: `Yes` or `No`.
Likert	2	Select exactly one of five ordered labels: Strongly disagree, Disagree, Neither agree nor disagree, Agree, Strongly agree.
Rating	2	Integer from 1 through 5.
Integer	2	Whole-number answer within optional author-defined minimum and maximum.
Date	2	Calendar date, stored without a time component or time zone.

The catalog has nine answer types. A dropdown is a presentation control for choice options, not a separate answer type. File upload and matrix/grid questions remain out of scope.

1.3 F-06 acceptance criteria (Sprint 2)

AC	Observable outcome
AC-F06-01	Each Sprint 2 type can be authored in a draft, rendered to a respondent, submitted and persisted using the correct answer representation.
AC-F06-02	Multi-choice accepts only options belonging to the question and enforces its configured minimum and maximum selections.
AC-F06-03	Long text accepts at most 10,000 characters; required values cannot be blank after trimming.
AC-F06-04	Yes/no accepts exactly one of the fixed `Yes` and `No` options.
AC-F06-05	Likert accepts exactly one of the five ordered labels defined in §1.2.
AC-F06-06	Rating accepts only integer values from 1 through 5.
AC-F06-07	Integer answers respect configured minimum and maximum bounds, when present.
AC-F06-08	Date answers accept a valid `YYYY-MM-DD` calendar date and persist without a time or time-zone shift.
AC-F06-09	The owner summary reports option counts/percentages, unlinked short- and long-text answers, rating/integer count, average, minimum and maximum, and date counts; it contains no respondent identity.

Part B continues in traceability.md, which contains the requirements matrix, coverage analysis and verification evidence rules.

Part B (continued): Requirements Traceability

Status note: every row is a **plan** until actual test evidence for the assessed commit is recorded in Part F. No requirement is marked verified in advance.

2. Traceability Matrix

Status values: **PLANNED** (not implemented), **IMPLEMENTED** (code exists, no mapped run is recorded), **VERIFIED** (a linked PR reports the mapped test passing; not a release-wide result), **PARTIAL** (coverage or evidence is incomplete), **FAILED**, **SKIPPED**, and **DEFERRED** (later sprint).

2.1 Functional requirements

Requirement	Summary	Feature	Sprint	Acceptance criteria	Planned verification	Status
FR-01	Login, logout and token revocation	F-01	1	AC-F01-01..05	STS-F01-01..05	VERIFIED
FR-02	Current user and role, no password hash exposure	F-01	1	AC-F01-03, AC-F01-06	STS-F01-03, STS-F01-06	VERIFIED
FR-03	Manage survey drafts	F-02	1	AC-F02-01..03, AC-F02-05	STS-F02-01..03, STS-F02-05	VERIFIED
FR-04	Author and edit draft questions	F-02	1	AC-F02-04, AC-F02-10	STS-F02-04, STS-F02-10	VERIFIED
FR-05	Publish survey	F-02	1	AC-F02-06, AC-F02-07	STS-F02-06, STS-F02-07	VERIFIED
FR-06	Close survey	F-02	1	AC-F02-08, AC-F02-11	STS-F02-08, STS-F02-11	VERIFIED
FR-07	Browse published surveys and load a survey form	F-03	1	AC-F03-01, AC-F03-02	STS-F03-01, STS-F03-02	VERIFIED
FR-08	Take and submit survey	F-03	1	AC-F03-03..07, AC-F03-09	STS-F03-03..07, STS-F03-09	VERIFIED
FR-09	Persisted confirmation and receipt retrieval	F-03	1	AC-F03-03, AC-F03-10	STS-F03-03, STS-F03-10	VERIFIED
FR-10	Aggregate summary (Sprint 1 formats)	F-04	1	AC-F04-01..04	STS-F04-01..04	VERIFIED
FR-11	Anonymous responses	F-05	2	AC-S2-01, AC-PRIV-01	STS-S2-01, STS-PRIV-01	DEFERRED
FR-12	Expanded question types and type-aware summary	F-06, Issue #9	2	AC-F06-01..09	Sprint 2 STS to be defined before implementation	DEFERRED
FR-13	CSV/JSON export	F-08	3	AC-S3-01	STS-S3-01	DEFERRED
FR-14	Eligibility allowlist	F-07	2	AC-S2-03	STS-S2-03	DEFERRED
FR-15	Scheduled close	F-07	2	AC-S2-04	STS-S2-04	DEFERRED
FR-16	Browse individual responses	F-08	3	AC-S3-02	STS-S3-02	DEFERRED

2.2 Business rules

Rule	Summary	Feature	Sprint	Acceptance criteria	Planned verification	Status
BR-01	One response per respondent per survey, including concurrency	F-03	1	AC-F03-04	STS-F03-04 (TDD)	VERIFIED
BR-02	Only PUBLISHED (and not past `closesAt`) accepts responses	F-03, F-07	1, 2	AC-F03-05, AC-S2-04	STS-F03-05, STS-S2-04	VERIFIED
BR-03	Required/type/option/text validation	F-03	1	AC-F03-06, AC-F03-07	STS-F03-06, STS-F03-07	VERIFIED
BR-04	Survey ownership boundary	F-02, F-04	1	AC-F02-09, AC-F04-03	STS-F02-09, STS-F04-03	VERIFIED
BR-05	Cross-response privacy	F-03, F-04	1	AC-F03-08, AC-F04-02	STS-F03-08, STS-F04-02	VERIFIED
BR-06	Publish structure requirements	F-02	1	AC-F02-07	STS-F02-07	VERIFIED
BR-07	Immutable structure after publication	F-02	1	AC-F02-10	STS-F02-10	VERIFIED
BR-08	Role and eligibility guard for submission	F-03, F-07	1, 2	AC-F03-09, AC-S2-03	STS-F03-09, STS-S2-03	VERIFIED
BR-09	CLOSED is terminal; invalid transitions rejected	F-02	1	AC-F02-11	STS-F02-11	VERIFIED
BR-10	Anonymous identity/answer separation	F-05	2	AC-S2-01, AC-PRIV-01	STS-S2-01, STS-PRIV-01	DEFERRED
BR-11	No respondent identity in aggregates or exports	F-04, F-08	1, 3	AC-F04-01, AC-S3-02	STS-F04-01, STS-S3-02	VERIFIED

2.3 Non-functional requirements

NFR	Target	Responsible feature	Sprint	Acceptance criterion	Verification	Status
NFR-SEC-01	Every protected endpoint rejects missing/invalid tokens with 401 and no data	F-01, cross-cutting	1	AC-SEC-01	STS-SEC-01 (parameterized route matrix)	PARTIAL
NFR-SEC-02	Every forbidden role/ownership operation returns 403 and leaves data unchanged	F-01..F-04	1	AC-SEC-02	STS-SEC-02 plus STS-F02-09, F03-08, F03-09, F04-02, F04-03	PARTIAL
NFR-SEC-03	bcrypt cost 10, no plaintext password, no hash in responses	F-01	1	AC-F01-06	STS-F01-06	PARTIAL
NFR-SEC-04	Logout revokes existing tokens	F-01	1	AC-F01-05	STS-F01-05	VERIFIED
NFR-PERF-01	p95 of `GET /api/surveys/:id/form` < 300 ms over 50 requests on the local development environment	F-03	1	AC-PERF-01	`server/scripts/benchmark-reads.ts`; result JSON in Part F	PLANNED
NFR-DATA-01	Foreign keys and unique constraints exist; data survives API and database restart	F-02, F-03	1	AC-DATA-01	Migration inspection, STS-F03-03, documented restart check	VERIFIED

Continued on next page

NFR	Target	Responsible feature	Sprint	Acceptance criterion	Verification	Status
NFR-PRIV-01	Anonymous responses have no identity link and no identity fields in output	F-05	2	AC-PRIV-01	STS-PRIV-01 database audit and response-shape assertions	DEFERRED

Deferred rows use `AC-S2-*`, `STS-S2-*`, `AC-S3-*` and `STS-S3-*` as scheduling placeholders. They are defined in the Sprint 2 and Sprint 3 contracts; they are not claimed as existing Sprint 1 IDs.

2.4 Cross-cutting test suites These supplemental suites cover criteria across more than one feature. Sprint 2 criteria remain deferred until their contracts and tests are defined.

Test ID	Scope	Linked acceptance criteria	Status
STS-U-01	Reject a draft with no questions	AC-F02-07	VERIFIED
STS-U-02	Reject a single-choice question with one option	AC-F02-07	VERIFIED
STS-U-03	Report an omitted required answer	AC-F03-06	VERIFIED
STS-U-04	Reject blank required and over-limit short text	AC-F03-06, AC-F03-07	VERIFIED
STS-U-05	Calculate response percentages and rounding	AC-F04-01	VERIFIED
STS-E2E-01	Complete owner-to-respondent survey journey and inspect summary	AC-F02-02, AC-F02-06, AC-F03-03, AC-F04-01	PLANNED
STS-E2E-02	Reject duplicate submission and cross-response access; verify token revocation	AC-F01-05, AC-F03-04, AC-F03-08	PLANNED

---

3. Coverage and Gap Analysis

3.1 Coverage summary

Requirement group	Total	Mapped	Sprint 1 executable	Later sprint
Functional requirements	16	16	10	6
Business rules	11	11	10	1
Non-functional requirements	7	7	6	1
Total	**34**	**34**	**26**	**8**

- No baseline requirement is unmapped.
- FR-11..FR-16, BR-10 and NFR-PRIV-01 have no Sprint 1 verification and are marked DEFERRED, not missing.
- BR-02 (published/closed survey), BR-08 (role guard), and BR-11 (aggregate privacy) have Sprint 1 portions reported as verified; scheduled close, eligibility, and export extensions remain deferred.
- Feature-level integration and unit tests exist for F-01 through F-04. The dedicated E2E suite, complete security route matrix, performance measurement, and release-wide test run are not recorded; see Part F.
- The previous draft's uncovered items were FR-04 (no edit endpoint), BR-07 (no way to modify a survey), and the eligibility behaviour required by the theme's Sprint 1 example. All three are now explicitly covered.

3.2 Approved baseline changes

Change	Affected requirements/features	Reason	Recorded in
Added owner list/detail/update and draft-delete endpoints	FR-03, FR-04, BR-07, F-02	The previous baseline promised editing but lacked supporting lifecycle endpoints, so the draft flow and BR-07 could not be fully tested	Part A §4; Part C API table; Part E F-02
Clarified draft lifecycle coverage in FR-03 and Part B	FR-03, F-02	F-02 already specified list, detail and draft deletion, but the requirements and traceability matrix did not identify those behaviors	Part A §4; Part B §2.1
Added `AnswerOption` and `numberValue`	FR-12, F-06	One selected option per answer cannot represent multi-select; ratings need a numeric field	Part C §2; Part A ADR-05
Expanded F-06 to the nine-type question catalog and fixed type-specific answer rules	FR-12, BR-03, BR-06, F-06	The previous baseline covered only multi-choice and rating; the expanded product needs explicit answer shapes, bounds and Sprint 2 allocation	Part A §§3-5; Part B §§1.2-1.3; Part C §§2, 8; Part D; GitHub Issues #1 and #9

Continued on next page

Change	Affected requirements/features	Reason	Recorded in
Added `SurveyEligibility`	FR-14, BR-08, F-07	Eligibility was missing from the baseline despite the theme's Sprint 1 example	Part A §4; Part C §2
Added `ResponseClaim`	BR-01, BR-10, F-05	A nullable `respondentId` does not enforce one response per person in anonymous mode	Part C §2.4
Added `User.tokenVersion`	FR-01, NFR-SEC-04, F-01	Logout must invalidate the token, not only clear browser state	Part A ADR-02; Part E F-01
Replaced the performance claim with a defined benchmark	NFR-PERF-01	"24 tests in 2.65 s" was not a latency measurement	Part A §6; Part F §5
Added `GET /api/health`	Deployment readiness	Container and cloud health checks need an unauthenticated probe	Part C API table
Added Docker Compose for the local application and PostgreSQL	Sprint 1 environment	Team decision: make clean-clone setup reproducible in the scaffold; keep CI/CD and GCP deployment in Lab 5	Part C §6.3; Part D; GitHub Issue #2

3.3 Deliberate gaps and limitations

Item	Status	Handling
Anonymous mode (F-05)	Not in Sprint 1	Contract and schema placeholders exist; implementation and privacy audit in Sprint 2
Multi-choice, long text, yes/no, Likert, rating, integer and date (F-06)	Not in Sprint 1	Sprint 2; Sprint 1 remains single-choice and short text
Eligibility allowlist (F-07)	Not in Sprint 1	Empty eligibility list means "all respondents" in Sprint 1; management in Sprint 2
Export (F-08)	Not in Sprint 1	Sprint 3
Rate limiting, refresh tokens, password reset	Not planned	Recorded as technical debt in Part F; outside the course baseline
Production-scale performance	Not claimed	NFR-PERF-01 has not been measured in the reviewed snapshot

3 Part C: System-Level Software Design Specification (SDS)

Status note: this is a design document. It does not claim that the system is implemented or deployed. Implementation and execution evidence belong to Part F.

---

1. Architecture and Component Responsibilities

1.1 Architecture style

SurveyHub is a **three-tier modular monolith**:

The React SPA sends JSON requests to the Express API. Authentication and role/ownership checks run before controllers call domain services; Prisma persists data in PostgreSQL. The client and API share one origin in the local deployment.

A modular monolith is deliberate: one deployable API container, one database, clear module boundaries, and no distributed-system overhead for a two-person course project.

1.2 Stack decision

Layer	Technology	Reason
Client	React 18, Vite, TypeScript, Tailwind CSS, React Router, Axios	Same stack as Labs 1-4; fast local development; static build
API	Node.js 20+, Express 4, TypeScript, Zod	Simple REST, typed validation, easy Supertest integration
ORM	Prisma 6	Typed schema, migrations, seed, PostgreSQL constraints
Database	PostgreSQL 16+ (local verification on 18.1)	Relational integrity, transactions, mature Docker and Cloud SQL support
Auth	`jsonwebtoken`, `bcryptjs`	Standard JWT and password hashing; no external identity provider
Tests	Jest, ts-jest, Supertest, Playwright	Unit, API/DB integration and E2E at the required levels
Local container workflow	Docker Compose for the application and PostgreSQL (Sprint 1); GitHub Actions and GCP Cloud Run + Cloud SQL (Lab 5)	Reproducible local development first, course deployment later

1.3 Layer responsibilities

Client tier

- Renders role-aware routes and forms.
- Performs only convenience validation; the server repeats every check.

- Holds the JWT and current-user state.
- Shows loading, empty, validation and error states.
- Never decides authorization from a hidden button. Hiding a control is not a security boundary.

Application tier

- `authenticateJWT`: verifies signature and expiry, loads the user, compares `tokenVersion`.
- `requireRole`: rejects a role that is not allowed for the route.
- Ownership checks live in services, because ownership depends on the record, not only the route.
- Controllers validate input with Zod and map errors to one JSON envelope.
- Services enforce business rules and run multi-row writes in a Prisma transaction.
- Prisma is the only component that talks to PostgreSQL.

Data tier

- Primary keys, foreign keys, unique constraints and indexes enforce structural integrity.
- Transactions protect response + answers + claim creation.
- Migrations are versioned in `server/prisma/migrations/`.
- Seed data is reproducible and idempotent.

1.4 Business-rule enforcement map

Rule	Service layer	Database layer
BR-01 one response per respondent	Pre-check claim, catch `P2002`	`ResponseClaim(survey_id, user_id)` unique; identified `SurveyResponse(survey_id, respondent_id)` unique
BR-02 state guard	Check `status = PUBLISHED` and `closesAt`	No direct constraint; status is an enum
BR-03 answer validation	Validate required, exact value shape, option ownership, type-specific text length, rating and integer bounds, date validity	`Answer(response_id, question_id)` unique; FKs
BR-04 ownership	Compare `survey.ownerId` with current user	FK `owner_id`
BR-05 cross-response privacy	Compare `response.respondentId` with current user	FK `respondent_id`

Continued on next page

Rule	Service layer	Database layer
BR-06 publish structure	Validate question count, type-specific options, selection/integer bounds and prompts	Service-level only
BR-07 immutable structure	Reject update when status is not DRAFT	Service-level only
BR-08 eligibility	Role check; optional allowlist check	`SurveyEligibility(survey_id, user_id)` unique
BR-09 terminal state	State-transition check	Enum status
BR-10 anonymous separation	Write `respondentId = NULL`; create claim without response link	Nullable `respondent_id`; separate `response_claims` table
BR-11 result privacy	Build summary without identity fields	No identity columns in aggregate queries

---

2. Data Model

2.1 Entity relationships

Users own surveys and may be linked to identified responses, eligibility entries, or response claims. Surveys contain questions and receive responses; questions contain options. Responses contain answers, and answers reference selected options through the extttAnswerOption join table.

2.2 Design decisions

1. **Answers are typed.** One `Answer` row exists per question per response. Single-choice, multi-choice, yes/no and Likert answers reference options through `AnswerOption`; short and long text use `textValue`; rating and integer answers use `numberValue`; date answers use `dateValue`. The service validates that each answer has the value shape required by its question type.

2. **One answer row per question per response.** `Answer(response_id, question_id)` is unique. A multi-choice answer still has one `Answer` row and several `AnswerOption` rows.

3. **Duplicate prevention is separate from response storage.** `ResponseClaim` records that a user submitted a survey. In anonymous mode it has no link to the response, so the system can prevent a second submission without knowing which response belongs to the user.

4. **Eligibility is a table, not a boolean.** An empty eligibility list means ”all respondents” in Sprint 1. Populated lists restrict submission in Sprint 2.

5. **Closed is terminal.** `publishedAt`, `closesAt` and `closedAt` capture lifecycle history.

6. **No user deletion in scope.** Foreign keys use `Restrict` where deleting a user or survey would destroy audit-relevant data. Draft survey deletion is handled by the service and cascades only through structure that has no responses.

2.3 Key entities, fields and constraints

The schema is implemented in `server/prisma/schema.prisma`. Every entity has a UUID `id` primary key. This summary lists the remaining fields and relations needed to understand the design; the repository schema remains the source of truth.

Entity	Key fields	Relations and constraints
User	`email`, `passwordHash`, `fullName`, `role`, `tokenVersion`, `createdAt`, `updatedAt`	Email is unique; owns surveys and may have responses, claims, and eligibility entries
Survey	`ownerId`, `title`, `description`, `status`, `mode`, `publishedAt`, `closesAt`, `closedAt`, `createdAt`, `updatedAt`	Belongs to an owner; indexes support owner/status and status/closing-time queries
SurveyEligibility	`surveyId`, `userId`, `createdAt`	Links a survey and user; unique on `(surveyId,userId)`
Question	`surveyId`, `prompt`, `questionType`, `isRequired`, `orderIndex`, `minSelections`, `maxSelections`, `numberMin`, `numberMax`	Belongs to a survey; indexed by survey and order
Option	`questionId`, `label`, `orderIndex`	Belongs to a question; label is unique within that question
SurveyResponse	`surveyId`, nullable `respondentId`, `submittedAt`	Belongs to a survey; identified response uniqueness is enforced per survey/respondent
ResponseClaim	`surveyId`, `userId`, `createdAt`	Unique per survey/user; prevents repeat submissions, including in anonymous mode
Answer	`responseId`, `questionId`, `textValue`, `numberValue`, `dateValue`	Unique per response/question; references its response and question
AnswerOption	`answerId`, `optionId`	Join between an answer and selected options; unique on `(answerId,optionId)`

Foreign keys cascade for survey structure, answers, claims, and eligibility entries where defined. Deletion is restricted for owners, surveys with responses, respondents with responses, questions with answers, and options in use.

2.4 Constraints added by SQL migration

Prisma cannot express every check, so the initial migration adds:

```
ALTER TABLE surveys
  ADD CONSTRAINT survey_title_length CHECK (char_length(title)
     BETWEEN 1 AND 120);

ALTER TABLE questions
  ADD CONSTRAINT question_prompt_length CHECK (char_length(prompt)
     BETWEEN 1 AND 500),
  ADD CONSTRAINT question_selection_bounds CHECK (
    min_selections IS NULL OR max_selections IS NULL OR
    (min_selections >= 1 AND min_selections <= max_selections)
  ),
  ADD CONSTRAINT question_integer_bounds CHECK (
    number_min IS NULL OR number_max IS NULL OR number_min <=
       number_max
  );
```

```
ALTER TABLE answers
  ADD CONSTRAINT answer_text_length CHECK (text_value IS NULL OR
     char_length(text_value) <= 10000);
```

These constraints are defence in depth. The service still returns friendly validation messages; the database is the last line of defence. Type-specific limits remain service-validated: Sprint 1 short text is limited to 2,000 characters, Sprint 2 long text to 10,000, rating to 1-5, and integer answers to their question's configured bounds.

2.5 Migration and seed strategy

- `npm run db:migrate` → `prisma migrate dev` locally; `prisma migrate deploy` in deployment.
- Migration files are committed. No manual schema edits in the database.
- `npm run db:seed` is idempotent: it uses `upsert` for users, surveys, questions and options.
- Seed data is synthetic and creates no responses or claims. Tests create their own response fixtures so counts are predictable. No real student data is used.

The seed creates two synthetic Survey Owner accounts and two synthetic Respondent accounts. Demo credentials are not repeated here.

Seeded surveys

Survey	Status	Questions	Purpose
CPE 334 Midterm Feedback	PUBLISHED	Single-choice + short text	Normal journey
Internal Team Survey Draft	DRAFT	Single-choice	Draft/publish tests
Owner 2 Course Pulse	PUBLISHED	Rating (Sprint 2 placeholder)	Cross-owner tests

2.6 Test database

- Integration tests use `DATABASE_URL_TEST` pointing to `surveyhub_test`.
- A global setup runs migrations and truncates tables in dependency order: `answer_options -> answers -> response_claims -> survey_responses -> options -> questions -> survey_eligibility -> surveys -> users`.
- Tests run with `jest --runInBand` so database state is deterministic.
- No test mocks Prisma for persistence or permission tests. Mocks are allowed only for isolated unit tests of pure functions.

---

3. UI Structure and Behaviour

3.1 Routes

Route	Role	Purpose
`/login`	Public	Login form
`/owner/surveys`	SURVEY_OWNER	Owner survey list with status badges and actions
`/owner/surveys/new`	SURVEY_OWNER	Create a draft survey and questions
`/owner/surveys/:id/edit`	SURVEY_OWNER	Edit a draft survey
`/owner/surveys/:id/summary`	SURVEY_OWNER	Aggregate summary
`/surveys`	RESPONDENT	Published survey catalogue
`/surveys/:id/take`	RESPONDENT	Survey form
`/surveys/:id/confirmation/:responseId`	RESPONDENT	Submission receipt
`/`	Any authenticated	Redirect by role
`*`	Any	Not-found page

`ProtectedRoute` checks authentication and role on the client for navigation only. Every API request is independently authorized on the server.

3.2 Shared UI states

State	Behaviour
Loading	Skeleton or spinner; submit button disabled while a request is pending
Empty	Explanatory text and a relevant action, e.g. "No surveys yet - create your first survey"
Validation	Inline field message, `aria-invalid`, focus moves to the first invalid field
Error banner	Red banner with the server's safe message; `aria-live="polite"`
Success	Confirmation card or toast; no raw error codes shown to end users
Access denied	A 403 page with a link back to the user's own area; no record data is rendered
Not found	A 404 page; does not reveal whether a record exists for another user

3.3 Accessibility and usability conventions

- Every input has a visible label; placeholders are not labels.
- Keyboard navigation works for the builder, radio groups and form submission.
- Colour is not the only status indicator; badges include text.
- Error messages describe what to fix.
- The layout is responsive down to 375 px.
- Destructive actions (delete draft, close survey) require confirmation.

4. API Design and Conventions

4.1 Envelope

Success:

```
{
  "success": true,
  "data": {}
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "The request contains invalid fields.",
    "details": [
      { "field": "title", "message": "Title is required." }
    ]
  }
}
```

Rules:

- `message` is safe for end users and never contains SQL, stack traces or secrets.
- `details` is optional and contains field-level validation messages.
- HTTP status codes carry the class of error; the `code` is stable for tests and clients.
- All timestamps are ISO 8601 UTC strings. Date answers use the date-only `YYYY-MM-DD` form.
- All ids are UUID strings.
- JSON request bodies are limited to 100 kB.

4.2 Status codes

Code	Meaning
200	Successful read or update
201	Resource created
204	Successful delete with no body
400	Validation or invalid state transition
401	Missing, invalid, expired or revoked token
403	Authenticated but not allowed (role, ownership, eligibility, cross-response)
404	Record not found
409	Duplicate submission

Continued on next page

Code	Meaning
500	Unexpected server error; generic message only
503	Database readiness check failed

4.3 Error codes

Code	HTTP	Used when
`VALIDATION_ERROR`	400	Malformed body or invalid field
`INVALID_CREDENTIALS`	401	Wrong email or password
`TOKEN_REQUIRED`	401	No Authorization header
`INVALID_TOKEN`	401	Bad signature, expired or malformed token
`TOKEN_REVOKED`	401	Token version no longer matches the user
`FORBIDDEN_ROLE`	403	Role not allowed for the operation
`FORBIDDEN_OWNERSHIP`	403	Record belongs to another owner
`FORBIDDEN_CROSS_RESPONDENT_ACCESS`	403	Respondent requests another respondent's receipt
`NOT_ELIGIBLE`	403	Respondent is not on the allowlist (Sprint 2)
`NOT_FOUND`	404	Unknown id
`DUPLICATE_SUBMISSION`	409	Claim already exists or unique violation
`SURVEY_NOT_ACCEPTING_RESPONSES`	403	Survey is draft, closed or past `closesAt`
`MISSING_REQUIRED_QUESTIONS`	400	Required answers absent
`INVALID_ANSWER`	400	Wrong value shape, foreign option, duplicate question, invalid type-specific bound or date
`INVALID_QUESTION_STRUCTURE`	400	Publish requirements not met
`SURVEY_NOT_EDITABLE`	400	Update/delete attempted after publication
`INVALID_STATE_TRANSITION`	400	Publish/close transition not allowed
`SERVICE_UNAVAILABLE`	503	Database connectivity check failed at the health endpoint
`INTERNAL_ERROR`	500	Unhandled error

4.4 Endpoint table

All endpoints except login and health require `Authorization: Bearer <token>`.

#	Method	Path	Auth	Allowed role	Purpose
1	POST	`/api/auth/login`	Public	-	Authenticate and return JWT
2	GET	`/api/auth/me`	Token	Any	Current user identity and role
3	POST	`/api/auth/logout`	Token	Any	Increment `tokenVersion` and revoke tokens
4	GET	`/api/health`	Public	-	Liveness/readiness probe
5	GET	`/api/surveys`	Token	SURVEY_OWNER	List own surveys
6	POST	`/api/surveys`	Token	SURVEY_OWNER	Create draft survey with questions
7	GET	`/api/surveys/:id`	Token	SURVEY_OWNER (owner)	Get own survey detail
8	PUT	`/api/surveys/:id`	Token	SURVEY_OWNER (owner)	Replace draft metadata and questions
9	DELETE	`/api/surveys/:id`	Token	SURVEY_OWNER (owner)	Delete a draft survey
10	PATCH	`/api/surveys/:id/publish`	Token	SURVEY_OWNER (owner)	Publish a valid draft

Continued on next page

#	Method	Path	Auth	Allowed role	Purpose
11	PATCH	/api/surveys/:id/close	Token	SURVEY_OWNER (owner)	Close a published survey
12	GET	/api/surveys/:id/summary	Token	SURVEY_OWNER (owner)	Aggregate results
13	GET	/api/surveys/active	Token	RESPONDENT	List published surveys and submission state
14	GET	/api/surveys/:id/form	Token	RESPONDENT	Get the form for a published survey
15	POST	/api/surveys/:id/responses	Token	RESPONDENT	Submit one response
16	GET	/api/surveys/:id/responses/:responseId	Token	RESPONDENT (own)	Get own submission receipt

Route registration order: `/api/surveys/active` must be registered before `/api/surveys/:id` so the literal path is not captured as an id.

GET /api/health This public endpoint is used by local containers and deployment probes to check API and database readiness. It requires no authentication and has no request body. The handler runs `SELECT 1` and exposes no table data, credentials, version, host, environment variables or stack trace.

Success `200`:

```
{
  "success": true,
  "data": { "status": "ok", "database": "up" }
}
```

Database check failure `503`:

```
{
  "success": false,
  "error": {
    "code": "SERVICE_UNAVAILABLE",
    "message": "Database connection failed."
  }
}
```

4.5 Validation approach

- Zod schemas validate body, params and query at the controller boundary.
- Unknown fields are rejected or stripped; the contract uses strict objects.
- Service-level domain validation runs after shape validation.
- Validation errors return field-level details and never echo the submitted password.
- `PUT /api/surveys/:id` replaces the full question set atomically inside a transaction. It is allowed only when the survey is `DRAFT`.

5. Authentication and Authorization

5.1 Password handling

- Passwords are hashed with `bcryptjs`, cost factor 10.
- `passwordHash` is never returned by any endpoint.
- Seed passwords are hashed at seed time, never stored as plaintext.
- Login compares the submitted password with the hash using a constant-time bcrypt comparison.

5.2 Token lifecycle

1. Login returns a JWT signed with `JWT_SECRET` (HS256), expiring after `JWT_EXPIRES_IN` (default 8h).
2. Payload: `{ sub: userId, role, ver: tokenVersion }`.
3. Every protected request verifies the signature and expiry, loads the user, and compares `ver` with the database value.
4. Logout increments `User.tokenVersion`. Every token issued before logout now fails with `401 TOKEN_REVOKED`.
5. There are no refresh tokens. After expiry the user logs in again.
6. The client stores the token in `localStorage` for refresh persistence and clears it on 401 or logout. This is an accepted trade-off for the course; XSS risk is reduced with a strict CSP and no `dangerouslySetInnerHTML`. HttpOnly cookies are recorded as future work.

5.3 Role and ownership matrix

Operation	Unauthenticated	RESPONDENT	SURVEY_OWNER (non-owner)	SURVEY_OWNER (owner)	Respondent (other user's receipt)
Login	Allowed	Allowed	Allowed	Allowed	Allowed
`/auth/me`, `/auth/logout`	401	Allowed	Allowed	Allowed	Allowed
List/create own surveys	401	403	Allowed	Allowed	403
Get/update/delete own draft	401	403	403	Allowed	403
Publish/close own survey	401	403	403	Allowed	403
View own summary	401	403	403	Allowed	403
Browse active surveys	401	Allowed	403	403	Allowed
Get published form	401	Allowed	403	403	Allowed

Continued on next page

Operation	Unauthenticated	RESPONDENT	SURVEY_OWNER (non-owner)	SURVEY_OWNER (owner)	Respondent (other user's receipt)
Submit response	401	Allowed, if eligible and not already submitted	403	403	Allowed
Get own receipt	401	Allowed	403	403	403

Key correction from the previous draft: owners are **not** allowed to open or submit survey forms. Roles are exclusive.

5.4 Authorization rules

- Role guard first: wrong role → `403 FORBIDDEN_ROLE`.
- Ownership check second: wrong owner → `403 FORBIDDEN_OWNERSHIP`.
- For receipt access: if the response belongs to another user → `403 FORBIDDEN_CROSS_RESPONSE_ACCES`
- For unknown ids → `404 NOT_FOUND`.
- A forbidden response never includes record data.
- UI hiding is not treated as authorization; tests call the API directly.

---

6. Configuration and Delivery Design

6.1 Environment variables

```
# Server
PORT=4000
NODE_ENV=development

# Database
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/
   surveyhub?schema=public"
DATABASE_URL_TEST="postgresql://postgres:postgres@localhost:5432/
   surveyhub_test?schema=public"

# Authentication
JWT_SECRET="replace-with-a-long-random-value"
JWT_EXPIRES_IN="8h"

# Client
CLIENT_ORIGIN="http://localhost:5173"
VITE_API_BASE_URL="http://localhost:4000/api"
```

The example uses the default port 5432. The actual port is whatever the local PostgreSQL instance uses; it is configured only through `DATABASE_URL` and `DATABASE_URL_TEST`.

Rules:

- `.env` is ignored by Git. `.env.example` contains placeholders only.
- The server validates required variables at startup and refuses to start in production with a missing or default `JWT_SECRET`.
- CORS allows only `CLIENT_ORIGIN`.
- `helmet` sets security headers; JSON body size is limited.
- No secret is logged.

6.2 Local reproduction

```
npm install
cp server/.env.example server/.env
npm run db:migrate
npm run db:seed
npm run dev
```

The client runs on `http://localhost:5173`; the API runs on `http://localhost:4000`.

6.3 Docker Compose local environment (Sprint 1)

Sprint 1 Issue #2 provides a root-level Compose setup with one application service and one PostgreSQL service.

- A multi-stage application image builds the React client and Express/TypeScript server. In the container environment, Express serves `/api` and the built client from the same origin.
- PostgreSQL has a health check. Compose starts the application only after the database reports healthy. A fresh database volume initialization creates both `surveyhub` and the separate `surveyhub_test` database.
- A named database volume preserves data when containers restart or are recreated. Removing the volume is a separate destructive operation and is not part of the normal reset workflow.
- `DATABASE_URL` targets the development database and `DATABASE_URL_TEST` targets the distinct test database. Inside Compose, both use the database service hostname; host-run tests use the published local port. Automated tests must never point at the seeded development database.
- Migration and seed commands are explicit, repeatable commands documented in the root README; application startup does not silently reset or seed persistent data.
- The application exposes public `/api/health` for readiness checks.
- Compose uses placeholder/local values only. Real secrets are kept outside Git.

6.4 CI/CD and GCP delivery plan (Lab 5)

- GitHub Actions on pull requests: install, lint, typecheck, migrate the test database, run tests and build.
- A required check blocks merge when it fails.
- The release workflow on `main` builds and pushes an image, runs migrations, and deploys to Cloud Run.
- Cloud Run uses the `PORT` environment variable and `/api/health` probe. Cloud SQL provides production database durability; secrets come from GCP Secret Manager or Cloud Run environment variables, never from the image.
- Deployment records the image tag and commit SHA and is repeatable from the same commit.

---

7. Anonymous-Mode Privacy Design (Sprint 2)

This section exists now because the schema must not need a breaking migration later.

7.1 Separation

- `Survey.mode = ANONYMOUS`.
- `SurveyResponse.respondentId = NULL`.
- `ResponseClaim` stores `(surveyId, userId)` only. It has no `responseId` column.
- The system knows **that** a user submitted, not **which** response is theirs.
- No IP address, user agent or other request metadata is stored with a response.
- The response id is a random UUID and is not derivable from the user id.

7.2 What the owner sees

- Aggregate counts and percentages.
- Rating and integer statistics and date counts.
- Unlinked short- and long-text answers.
- No respondent names, emails or ids.
- Export for an anonymous survey contains aggregates and unlinked text only.

7.3 Residual risk

A database administrator could correlate `ResponseClaim.createdAt` with `SurveyResponse.submitted` To reduce this, the claim row does **not** store a timestamp in Sprint 2; only the unique key is kept. Even so, an administrator with access to database transaction logs could correlate activity. The product does not claim cryptographic unlinkability against a privileged database operator. This limitation is disclosed rather than overclaimed.

---

8. Forward-Compatibility Checklist

Future need	Present in Sprint 1 schema	Later work
Expanded question types	`QuestionType`; `AnswerOption`; `textValue`; `numberValue`; `dateValue`; `minSelections`, `maxSelections`, `numberMin`, `numberMax`	Sprint 2 validation, authoring UI, response UI and summary for multi-choice, long text, yes/no, Likert, rating, integer and date
Anonymous mode	`SurveyMode.ANONYMOUS`, nullable `respondentId`, `ResponseClaim` without response link	Endpoints, privacy tests, UI
Eligibility	`SurveyEligibility` table	Management API and UI
Scheduled close	`closesAt`	Date picker and server check
Export	Existing models and summary service	CSV/JSON serializers
Raw response browser	Identified responses already linked	Owner-only endpoints and UI

---

9. Traceability

- Requirements: Part A.
- Feature allocation: Part B.
- Sprint plan: Part D.
- Sprint 1 contracts and tests: Part E.
- Verification evidence: Part F.

No design decision in this document is considered implemented until the corresponding code and tests exist on the assessed commit.

4 Part D: Three-Sprint Product Development Plan

Planning baseline dated 5 November 2026. The sprint goals and outcomes below are targets; current Sprint 1 implementation and verification evidence appears in Part F.

---

1. Product Vision and Sprint Allocation

SurveyHub must be a complete survey product by 13 December 2026: owners create and publish surveys, eligible respondents submit validated answers, owners read trustworthy aggregates, and later sprints add anonymous mode, richer question types, eligibility control, export and raw-response browsing.

---

2. Sprint Matrix

Sprint	Dates	Goal	Features	Requirements	Target outcome
Sprint 1	5-8 Nov 2026	Working vertical slice through UI, API and database with authentication, ownership and tested rules	F-01, F-02, F-03, F-04	FR-01..FR-10; BR-01..BR-09; BR-11 (aggregate privacy portion); NFR-SEC-01..04, NFR-PERF-01, NFR-DATA-01	Feature PRs merged to `sprint-1-staging`; approved release PR to `main`, tag `v0.1.0-sprint1`, tests and demo evidence, D1 PDF
Sprint 2	9-29 Nov 2026	Complete the core product: anonymous mode, advanced question types, eligibility, deployment and independent UAT	F-05, F-06 (Issue #9), F-07	FR-11, FR-12, FR-14, FR-15; BR-02 (scheduled close), BR-08 (allowlist), BR-10, NFR-PRIV-01	Deployed integrated release candidate on GCP; D2; Lab 5 and Lab 6 complete
Sprint 3	30 Nov-13 Dec 2026	Finish analytics/export, harden, fix defects and release the complete product	F-08 plus hardening	FR-13, FR-16; BR-11 (raw/export policy)	Final release, Lab 7 change, final report and video

---

3. Sprint 1 Scope

3.1 In scope

Feature	Included behaviour
F-01 Auth & RBAC	Login, `/me`, logout with `tokenVersion` revocation, bcrypt cost 10, role guard, 401/403 errors
F-02 Survey lifecycle	List own surveys, create draft, get detail, replace draft questions, delete draft, publish, close, ownership and immutability guards

Continued on next page

Feature	Included behaviour
F-03 Response rules	Browse published surveys, load form, submit one response, persisted receipt, duplicate guard, state guard, required/type/option validation, cross-response privacy
F-04 Aggregate summary	Total responses, answered count per question, option counts and percentages, unlinked text answers, owner-only access, empty state
E2E	One normal journey and one rejection journey

3.2 Out of scope for Sprint 1

- Anonymous mode (F-05) - Sprint 2.
- Multi-choice, long text, yes/no, Likert, rating, integer and date (F-06) - Sprint 2.
- Eligibility allowlist management and scheduled close (F-07) - Sprint 2.
- CSV/JSON export, charts, raw-response browser (F-08) - Sprint 3.
- CI/CD and GCP deployment - Lab 5, after D1. Docker Compose local setup is included in Sprint 1 Issue #2.
- Self-registration, password reset, email invitations - excluded.

3.3 Sprint 1 completion criteria

- All Sprint 1 acceptance criteria in Part E are implemented and verified, or explicitly disclosed as incomplete.
- The full test command runs on the release commit and counts are recorded.
- The normal journey, a rule rejection, a forbidden operation and logout are demonstrated with real evidence.
- The release is on `main`, tagged, and reproducible from the README.
- Part F contains no invented results.

---

4. Dependencies and Critical Path

4.1 Feature dependencies

Feature	Depends on	Reason
F-01	None	Identity and role are the foundation for every protected operation
F-02	F-01	Only an authenticated owner can manage surveys

Continued on next page

Feature	Depends on	Reason
F-03	F-01, F-02	A published survey must exist before a response can be submitted
F-04	F-01, F-02, F-03	Summary needs stored responses
F-05	F-01, F-03, F-04	Anonymous mode changes response storage and summary output
F-06	F-02, F-03, F-04	Authoring, submission and summary must all understand the new types
F-07	F-02, F-03	Eligibility and scheduled close modify publish and submit rules
F-08	F-04, F-06	Export and charts depend on summary data and question types

4.2 Critical path for Sprint 1

The critical path is **schema** → **auth** → **lifecycle** → **responses** → **verification**. A delay in the schema delays everything, so the schema and migration are completed before UI work.

4.3 Sprint 1 issue map

Issue	Work	Feature/contract
#1	Scope, requirements, feature inventory, design and Sprint 1 contracts	Parts A-E
#2	Application/database scaffold, migrations, seed and Docker Compose	Foundation
#3	Authentication, sessions and RBAC	F-01
#4	Survey lifecycle and authoring	F-02
#5	Survey response submission and business rules	F-03
#6	Owner aggregate summary	F-04
#7	Integration, verification evidence, release and deliverable	Part F and release

5 Part E: Sprint 1 Feature Contracts and Software Test Specifications

Contract status: this is the versioned Sprint 1 baseline. Acceptance criteria and STS define expected behavior; test outcomes and remaining evidence gaps are recorded in Part F.

---

1. Product Definition of Done (DoD)

A Sprint 1 feature is Done only when all of the following are true and evidenced:

1. **Behaviour:** every acceptance criterion is implemented through the UI, API and database.
2. **Rules:** business rules are enforced on the server, not only in the client.
3. **Tests:** planned unit, API/database integration and selected E2E tests exist and run.
4. **Negative cases:** boundary, invalid and forbidden cases are tested, not only happy paths.
5. **Persistence:** data is written to PostgreSQL and can be retrieved after a reload and a restart.
6. **Review:** a teammate reviewed the PR and left a specific technical check, question or finding.
7. **Integration:** the feature was merged into `sprint-1-staging` and the full suite passed there.
8. **Documents:** the contract, API shape and README are current.
9. **Disclosure:** any failed, skipped or incomplete work is listed in Part F.
10. **Evidence:** test output, screenshots and commit ids are real and identify the same version.

A feature is **not** Done because an AI agent says so, because a screenshot exists, or because an old test run passed.

Part E - F-01 Authentication, Session and RBAC

2. Contract F-01: Authentication, Session and RBAC

2.1 Scope and linked requirements

- **Requirements:** FR-01, FR-02
- **NFRs:** NFR-SEC-01, NFR-SEC-02, NFR-SEC-03, NFR-SEC-04
- **Feature:** F-01
- **Dependencies:** none
- **Sprint:** 1

2.2 Behaviour

- A seeded user logs in with email and password.
- The server normalizes the email (trim, lowercase), verifies the bcrypt hash and returns a JWT.
- The token contains `sub`, `role` and `ver` (token version) and expires after 8 hours.
- Every protected request verifies the token, loads the user and compares `ver`.
- Logout increments `tokenVersion`, revoking every existing token for that account.
- `GET /api/auth/me` returns the current identity and role.
- Passwords are hashed with bcrypt cost 10. No endpoint returns `passwordHash`.
- There is no self-registration, password reset or refresh token in this sprint.

2.3 UI behaviour `/login`

- Email and password fields with visible labels.
- Client-side validity checks show inline field messages, set `aria-invalid` and focus the first invalid field; server validation details use the same behavior.
- Client-side check for a non-empty email and password; server remains authoritative.
- Submit button disabled and spinner shown while the request is pending.
- `401` shows "Invalid email or password." in an alert banner with `aria-live`.
- Success stores the token, loads the current user and redirects:
  - `SURVEY_OWNER` → `/owner/surveys`
  - `RESPONDENT` → `/surveys`
- An expired or revoked token on any request clears the session and redirects to `/login`.

- `/` waits for the current-user check, then redirects authenticated users by role and unauthenticated users to `/login`, as specified in system SDS §3.1.

Navbar

- Shows full name and role.
- "Log out" calls the API, clears local state and redirects to `/login`.
- If the logout request fails, the client still clears local state; the server-side revocation is tested separately.

2.4 Data specification Table `users`:

Field	Type	Constraint
`id`	UUID	Primary key
`email`	text	Unique, stored lowercase
`password_hash`	text	bcrypt cost 10, `$2b$10$` prefix
`full_name`	text	1-120 characters
`role`	enum	`SURVEY_OWNER` or `RESPONDENT`
`token_version`	integer	Default 0, incremented on logout
`created_at`, `updated_at`	timestamptz	Automatic

No schema change is needed for RBAC beyond this table.

2.5 API specification

POST /api/auth/login (public) Request:

```
{ "email": "owner1@survey.local", "password": "<seeded-password>" }
```

Success 200:

```
{
  "success": true,
  "data": {
    "token": "<jwt>",
    "user": {
      "id": "<uuid>",
      "email": "owner1@survey.local",
      "fullName": "Course Coordinator",
      "role": "SURVEY_OWNER"
    }
  }
}
```

Errors:

- `400 VALIDATION_ERROR` - missing or malformed email/password, with field-level details that never include submitted values, as specified in system SDS §4.5.

- `401 INVALID_CREDENTIALS` - unknown email or wrong password. The message is identical for both to avoid account enumeration; unknown emails still perform a bcrypt comparison against a fixed dummy hash so response timing does not expose whether an account exists.

GET /api/auth/me (authenticated) Success 200:

```
{
  "success": true,
  "data": {
    "id": "<uuid>",
    "email": "resp1@survey.local",
    "fullName": "Student One",
    "role": "RESPONDENT"
  }
}
```

Errors: `401 TOKEN_REQUIRED`, `401 INVALID_TOKEN`, `401 TOKEN_REVOKED`.

POST /api/auth/logout (authenticated) Success 200:

```
{ "success": true, "data": { "message": "Successfully logged out." }
    }
```

Side effect: `token_version = token_version + 1`.
Errors: `401 TOKEN_REQUIRED`, `401 INVALID_TOKEN`, `401 TOKEN_REVOKED`.

2.6 Acceptance criteria

AC	Observable outcome
AC-F01-01	Valid credentials return `200`, a signed JWT and a user object without `passwordHash`.
AC-F01-02	A wrong password returns `401 INVALID_CREDENTIALS` and no token.
AC-F01-03	`GET /api/auth/me` with a valid token returns the id, email, full name and role stored in the database.
AC-F01-04	`POST /api/auth/logout` returns `200` and increments the user's `tokenVersion`.
AC-F01-05	A token used before logout returns `401 TOKEN_REVOKED` on the next protected request.
AC-F01-06	Every seeded password hash starts with `$2b$10$`; no plaintext password or hash appears in any API response.
AC-F01-07	Malformed login input returns `400 VALIDATION_ERROR` with field-level details and no submitted values.
AC-F01-08	Unauthenticated `GET /api/auth/me` and `POST /api/auth/logout` return `401 TOKEN_REQUIRED`.
AC-F01-09	A request to a role-protected route by the wrong role returns `403 FORBIDDEN_ROLE`.

2.7 Software Test Specification File: `server/tests/auth.test.ts`

Test ID	AC	Level	Precondition / input	Action	Expected result
STS-F01-01	AC-F01-01	Integration	Seeded owner account	POST login with correct password	`200`, token present, role `SURVEY_OWNER`, no `passwordHash`
STS-F01-02	AC-F01-02	Integration	Seeded owner account	POST login with wrong password	`401 INVALID_CREDENTIALS`, no token
STS-F01-03	AC-F01-03	Integration	Valid respondent token	GET `/api/auth/me`	`200`, email and role match the database
STS-F01-04	AC-F01-04	Integration	Valid token, `tokenVersion = 0`	POST logout, then read the user row	`200`; `tokenVersion = 1`
STS-F01-05	AC-F01-05	Integration	Token obtained before logout	Call `/api/auth/me` with the old token after logout	`401 TOKEN_REVOKED`
STS-F01-06	AC-F01-06	Integration	Seeded users / DB	Query `users.password_hash`; inspect login and `/me` responses	Every hash starts `$2b$10$`; no response contains a hash or plaintext
STS-F01-07	AC-F01-07	Integration	Malformed and missing login bodies	POST login	`400 VALIDATION_ERROR`, field details, no submitted values
STS-F01-08	AC-F01-08	Integration	No bearer token	GET `/api/auth/me`; POST `/api/auth/logout`	Both return `401 TOKEN_REQUIRED`
STS-F01-09	AC-F01-09	Integration	Valid token for each role	Call a SURVEY_OWNER-only route with owner and respondent tokens	Owner allowed; respondent gets `403 FORBIDDEN_ROLE`

2.8 Version history

Version	Date	Change	Status
v3	5 Nov 2026	Aligned the F-01 contract with team decisions and the shared Kanban issue	Superseded
v4	7 Nov 2026	Clarified SDS root-route and login-validation behavior, equal-cost unknown-user login checks, and issue #3 acceptance-test traceability	Current

Part E - F-02 Survey Lifecycle and Authoring

3. Contract F-02: Survey Lifecycle and Authoring

3.1 Scope and linked requirements

- **Requirements:** FR-03, FR-04, FR-05, FR-06
- **Business rules:** BR-04, BR-06, BR-07, BR-09
- **Feature:** F-02
- **Dependencies:** F-01
- **Sprint:** 1

3.2 Lifecycle

```
DRAFT --publish---> PUBLISHED --close---> CLOSED
  |                                        |
  +--delete (draft only)                   +-- terminal
```

Allowed operations:

Current status	Update	Delete	Publish	Close
DRAFT	Yes	Yes	Yes, if structure valid	No
PUBLISHED	No	No	No	Yes
CLOSED	No	No	No	No

Any other transition returns `400 INVALID_STATE_TRANSITION`.

3.3 UI behaviour `/owner/surveys`

- Lists only the owner's surveys with title, status badge, question count and response count.
- Actions depend on status:
  - DRAFT: Edit, Delete, Publish, View summary
  - PUBLISHED: Close, View summary
  - CLOSED: View summary
- Empty state: "No surveys yet. Create your first survey."
- Loading skeleton while fetching.

`/owner/surveys/new` and `/owner/surveys/:id/edit`

- Title (required, 1-120 characters), description (optional, $\leq$2,000 characters).
- Dynamic question list:
  - Type: single-choice or short text (Sprint 1).
  - Required toggle.

  - Prompt (1-500 characters).
  - For single-choice: 2-20 option labels, each 1-200 characters.
  - Reorder with up/down buttons.
  - Remove question with confirmation.
- Client prevents obvious invalid publish, but the server revalidates.
- Publish button shows a validation summary if the structure is invalid.
- Close opens a confirmation dialog.

`/owner/surveys/:id/summary` - defined in F-04.

3.4 Data specification Tables: `surveys`, `questions`, `options`.

- `surveys.title`: 1-120 characters (SQL check).
- `surveys.status`: enum, default `DRAFT`.
- `surveys.mode`: enum, default `IDENTIFIED`.
- `surveys.owner_id`: FK to `users.id`, `onDelete: Restrict`.
- `published_at`, `closed_at`: nullable timestamps.
- `questions.prompt`: 1-500 characters (SQL check).
- `questions.question_type`: enum.
- `questions.is_required`: default true.
- `questions.order_index`: non-negative; ordered by `orderIndex, id`.
- `options.label`: 1-200 characters; unique per question.
- `options.order_index`: ordered by `orderIndex, id`.

`PUT /api/surveys/:id` replaces the question set in a transaction:

1. Verify owner and `DRAFT`.
2. Validate the full new structure.
3. Delete existing questions (cascades options) and recreate them with stable order.
4. Update title and description.
5. Return the full updated survey.

No migration change is required for Sprint 1 beyond the initial schema.

3.5 API specification All routes require `SURVEY_OWNER`.

GET /api/surveys Success 200:

```
{
  "success": true,
  "data": [
    {
      "id": "<uuid>",
      "title": "CPE 334 Midterm Feedback",
      "status": "PUBLISHED",
      "mode": "IDENTIFIED",
      "questionCount": 2,
      "responseCount": 12,
      "createdAt": "2026-11-05T10:00:00.000Z",
      "updatedAt": "2026-11-05T10:30:00.000Z"
    }
  ]
}
```

Only surveys with `ownerId = currentUser.id`.

POST /api/surveys Request:

```
{
  "title": "CPE 334 Midterm Feedback",
  "description": "Please give honest feedback.",
  "questions": [
    {
      "prompt": "How satisfied are you with the course?",
      "questionType": "SINGLE_CHOICE",
      "isRequired": true,
      "options": [
        { "label": "Very satisfied" },
        { "label": "Neutral" },
        { "label": "Unsatisfied" }
      ]
    },
    {
      "prompt": "What should improve?",
      "questionType": "SHORT_TEXT",
      "isRequired": false
    }
  ]
}
```

Success 201: the full draft survey, `status: "DRAFT"`.
Errors: `400 VALIDATION_ERROR`, `403 FORBIDDEN_ROLE`.

GET /api/surveys/:id Success 200: full survey with ordered questions and options.
Errors: `403 FORBIDDEN_OWNERSHIP`, `404 NOT_FOUND`.

PUT /api/surveys/:id Same body as create. Replaces metadata and questions. Allowed only for `DRAFT`.

Success `200`: updated full survey.

Errors:

- `400 SURVEY_NOT_EDITABLE` - status is `PUBLISHED` or `CLOSED`.
- `400 VALIDATION_ERROR` - invalid structure.
- `403 FORBIDDEN_OWNERSHIP`, `404 NOT_FOUND`.

DELETE /api/surveys/:id Success `204` with no body. Cascades questions and options. Allowed only for `DRAFT`.

Errors: `400 SURVEY_NOT_EDITABLE`, `403 FORBIDDEN_OWNERSHIP`, `404 NOT_FOUND`.

PATCH /api/surveys/:id/publish Success `200`: status `PUBLISHED`, `publishedAt` set.

Errors:

- `400 INVALID_QUESTION_STRUCTURE` - no questions; a single-choice question has fewer than two options; empty prompt; duplicate option labels.
- `400 INVALID_STATE_TRANSITION` - not `DRAFT`.
- `403 FORBIDDEN_OWNERSHIP`, `404 NOT_FOUND`.

PATCH /api/surveys/:id/close Success `200`: status `CLOSED`, `closedAt` set.

Errors: `400 INVALID_STATE_TRANSITION` - not `PUBLISHED`; `403 FORBIDDEN_OWNERSHIP`, `404 NOT_FOUND`.

3.6 Acceptance criteria

AC	Observable outcome
AC-F02-01	`GET /api/surveys` returns only the current owner's surveys, with status and counts.
AC-F02-02	`POST /api/surveys` creates a `DRAFT` with ordered questions and options, stored in PostgreSQL.
AC-F02-03	`GET /api/surveys/:id` returns the owner's full survey detail.
AC-F02-04	`PUT /api/surveys/:id` updates a draft's title, description and questions atomically.
AC-F02-05	`DELETE /api/surveys/:id` deletes a draft and cascades its structure; no questions or options remain.
AC-F02-06	Publishing a valid draft returns `200` and sets `status = PUBLISHED` and `publishedAt`.
AC-F02-07	Publishing an invalid structure returns `400 INVALID_QUESTION_STRUCTURE` and leaves the status `DRAFT`.

Continued on next page

AC	Observable outcome
AC-F02-08	Closing a published survey returns `200` and sets `status = CLOSED` and `closedAt`.
AC-F02-09	A non-owner gets `403 FORBIDDEN_OWNERSHIP` for get, update, delete, publish and close; data is unchanged.
AC-F02-10	Updating or deleting a published/closed survey returns `400 SURVEY_NOT_EDITABLE`.
AC-F02-11	Publishing a published/closed survey or closing a draft/closed survey returns `400 INVALID_STATE_TRANSITION`.

3.7 Software Test Specification File: `server/tests/survey-lifecycle.test.ts`

Test ID	AC	Level	Precondition / input	Action	Expected result
STS-F02-01	AC-F02-01	Integration	Owner A has 2 surveys; Owner B has 1	GET `/api/surveys` as A	`200`, exactly A's 2 surveys
STS-F02-02	AC-F02-02	Integration	Owner A authenticated	POST a survey with 2 questions	`201`, `DRAFT`, persisted questions/options
STS-F02-03	AC-F02-03	Integration	Own draft with questions	GET `/api/surveys/:id`	`200`, ordered questions and options
STS-F02-04	AC-F02-04	Integration	Own draft	PUT with changed title/questions	`200`, new structure persisted, old questions removed
STS-F02-05	AC-F02-05	Integration / DB	Own draft with questions	DELETE	`204`; survey, questions and options absent
STS-F02-06	AC-F02-06	Integration	Valid draft	PATCH publish	`200`, `PUBLISHED`, `publishedAt` set
STS-F02-07	AC-F02-07	Integration	Draft with no questions; draft with 1 option	PATCH publish	`400 INVALID_QUESTION_STRUCTURE`; status remains `DRAFT`
STS-F02-08	AC-F02-08	Integration	Published survey	PATCH close	`200`, `CLOSED`, `closedAt` set
STS-F02-09	AC-F02-09	Integration	Survey owned by A; authenticated as B	GET, PUT, DELETE, publish, close	`403 FORBIDDEN_OWNERSHIP` for each; database unchanged
STS-F02-10	AC-F02-10	Integration	Published survey	PUT and DELETE	`400 SURVEY_NOT_EDITABLE` for both
STS-F02-11	AC-F02-11	Integration	Published survey; draft survey; closed survey	Publish again; close draft; close closed	`400 INVALID_STATE_TRANSITION` for each

Supporting unit tests for structure validation are in section 7.

Part E - F-03 Survey Taking, Submission and Response Rules

4. Contract F-03: Survey Taking, Submission and Response Rules

4.1 Scope and linked requirements

- **Requirements:** FR-07, FR-08, FR-09
- **Business rules:** BR-01, BR-02, BR-03, BR-05, BR-08
- **NFRs:** NFR-PERF-01, NFR-DATA-01
- **Feature:** F-03
- **Dependencies:** F-01, F-02
- **Sprint:** 1

4.2 Behaviour

- A respondent sees only published surveys.
- `hasSubmitted` is computed from `ResponseClaim`.
- When `hasSubmitted` is true, the active-survey item includes that respondent's `responseId` for opening their receipt; otherwise `responseId` is omitted.
- The form endpoint returns ordered questions and options for a published survey.
- Submission validates the whole payload before writing.
- The response, answers, selected options and claim are written in one transaction.
- A second submission is rejected with `409 DUPLICATE_SUBMISSION`; exactly one response remains.
- The receipt endpoint returns only the caller's own response.
- In Sprint 1, every authenticated respondent is eligible. The `SurveyEligibility` table is present but empty; Sprint 2 adds allowlist management.

4.3 UI behaviour `/surveys`

- Cards for published surveys with title, description and a status badge:
  - "Not submitted" → Start survey
  - "Completed" → View receipt, disabled start
- Empty state if no published surveys exist.
- Loading skeleton.

`/surveys/:id/take`

- Renders questions in `orderIndex` order.

- Single-choice: radio group.
- Short text: textarea with a character counter and a 2,000-character limit.
- Required questions show a visible "Required" marker.
- Submit disabled while pending.
- Client validates required answers for convenience; server revalidates.
- `409` shows "You have already submitted a response to this survey."
- `400 SURVEY_NOT_ACCEPTING_RESPONSES` shows "This survey is no longer accepting responses."
- `400 MISSING_REQUIRED_QUESTIONS` highlights the missing questions.

`/surveys/:id/confirmation/:responseId`

- Green confirmation card with response id and submitted time.
- A "View receipt" action re-fetches the persisted receipt from the API, proving persistence.

4.4 Data specification Tables: `survey_responses`, `answers`, `answer_options`, `response_claims`.

- `survey_responses.respondent_id`: nullable; non-null in Sprint 1 identified mode.
- `survey_responses.submitted_at`: server time.
- `response_claims(survey_id, user_id)` unique.
- `answers(response_id, question_id)` unique; one answer row per question per response.
- `answer_options(answer_id, option_id)` unique.
- `answers.text_value`: nullable, $\leq$10,000 characters in the forward-compatible schema; Sprint 1 short-text validation remains $\leq$2,000 characters.
- `answers.number_value`: nullable integer; Sprint 2 rating is 1-5 and integer questions use configured bounds.
- `answers.date_value`: nullable date-only value for Sprint 2 date questions.

Submission transaction:

```
BEGIN
  verify survey exists, is PUBLISHED, and (Sprint 2) is before
     closesAt
  verify role is RESPONDENT and eligibility if configured
  insert response_claim (unique survey_id + user_id)
  insert survey_response (respondent_id = current user)
  insert answers and answer_options
COMMIT
```

If the claim insert fails with `P2002`, return `409` and roll back. No partial response is stored.

4.5 API specification All routes require `RESPONDENT`.

GET /api/surveys/active Success 200:

```
{
  "success": true,
  "data": [
    {
      "id": "<uuid>",
      "title": "CPE 334 Midterm Feedback",
      "description": "Please give honest feedback.",
      "status": "PUBLISHED",
      "questionCount": 2,
      "hasSubmitted": false
    }
  ]
}
```

Only `PUBLISHED` surveys. `hasSubmitted` is true when a claim exists for the current user. If true, `responseId` is the current user's persisted response id, used by the catalogue's receipt link; it is omitted otherwise.

GET /api/surveys/:id/form Success 200: survey metadata plus ordered questions and options.

Errors: `400 SURVEY_NOT_ACCEPTING_RESPONSES` for draft/closed; `404 NOT_FOUND`.

POST /api/surveys/:id/responses Request:

```
{
  "answers": [
    { "questionId": "<uuid-1>", "selectedOptionIds": ["<uuid-option
       >"] },
    { "questionId": "<uuid-2>", "textValue": "More lab time, please
       ." }
  ]
}
```

Rules:

- Each question appears at most once. Duplicate question ids → `400 INVALID_ANSWER`.
- Unknown question ids → `400 INVALID_ANSWER`.
- Single-choice: exactly one selected option, belonging to that question.
- Short text: non-empty when required; trimmed; ≤2,000 characters.
- Required questions missing → `400 MISSING_REQUIRED_QUESTIONS` with the missing ids.
- A question must not mix option and text/number values.
- Optional unanswered questions may be omitted.

Success 201:

```
{
  "success": true,
  "data": {
    "responseId": "<uuid>",
    "surveyId": "<uuid>",
    "submittedAt": "2026-11-06T14:30:00.000Z",
    "message": "Survey response successfully recorded."
  }
}
```

Errors:

- 409 DUPLICATE_SUBMISSION
- 400 SURVEY_NOT_ACCEPTING_RESPONSES
- 400 MISSING_REQUIRED_QUESTIONS
- 400 INVALID_ANSWER
- 403 FORBIDDEN_ROLE
- 403 NOT_ELIGIBLE (Sprint 2)
- 404 NOT_FOUND

GET /api/surveys/:id/responses/:responseId Success 200: the caller's own receipt with answer content and `submittedAt`.

Errors:

- 403 FORBIDDEN_CROSS_RESPONSE_ACCESS when the response belongs to another user.
- 403 FORBIDDEN_ROLE for an owner account.
- 404 NOT_FOUND when the response id does not exist for that survey.

4.6 Acceptance criteria

AC	Observable outcome
AC-F03-01	`GET /api/surveys/active` returns published surveys only and a correct `hasSubmitted` flag; submitted items include only the current respondent's `responseId`.
AC-F03-02	`GET /api/surveys/:id/form` returns ordered questions and options for a published survey.
AC-F03-03	A valid submission returns 201 and persists one response, its answers, selected options and claim in PostgreSQL.
AC-F03-04	A second submission returns 409 DUPLICATE_SUBMISSION; exactly one response exists. Under two concurrent submissions, one succeeds and one gets 409.

Continued on next page

AC	Observable outcome
AC-F03-05	Submitting to a `DRAFT` or `CLOSED` survey returns `400 SURVEY_NOT_ACCEPTING_RESPONSES`.
AC-F03-06	Missing required answers return `400 MISSING_REQUIRED_QUESTIONS` with the missing question ids.
AC-F03-07	Wrong answer type, a foreign option, duplicate question ids or text over 2,000 characters returns `400 INVALID_ANSWER`.
AC-F03-08	A respondent requesting another respondent's receipt gets `403 FORBIDDEN_CROSS_RESPONSE_ACCESS` with no answer data.
AC-F03-09	A `SURVEY_OWNER` token submitting a response gets `403 FORBIDDEN_ROLE`.
AC-F03-10	The respondent can retrieve their own persisted receipt after a reload, with the same answers and timestamp.

4.7 Software Test Specification

File: `server/tests/response-submission.test.ts`

Test ID	AC	Level	Precondition / input	Action	Expected result
STS-F03-01	AC-F03-01	Integration	1 published, 1 draft, 1 closed survey; resp1 and resp2 have submitted to different surveys	GET `/api/surveys/active` as resp1	`200`, only published surveys, each with `hasSubmitted`; only resp1's own completed survey includes `responseId`
STS-F03-02	AC-F03-02	Integration	Published survey with 2 questions	GET `/api/surveys/:id/form`	`200`, ordered questions and options
STS-F03-03	AC-F03-03	Integration / DB	Published survey; resp1 has not submitted	POST valid answers	`201`; one response, 2 answers, option links and claim exist
STS-F03-04	AC-F03-04	Integration (TDD core)	resp1 already submitted; repeat sequentially, then two concurrent requests for resp2	POST again; POST twice in parallel	`409 DUPLICATE_SUBMISSION`; DB count remains 1; concurrent case yields one `201` and one `409`
STS-F03-05	AC-F03-05	Integration	Draft survey; closed survey	POST answers to each	`400 SURVEY_NOT_ACCEPTING_RESPONSES`
STS-F03-06	AC-F03-06	Integration	Required question omitted; optional omitted	POST	`400 MISSING_REQUIRED_QUESTIONS` with id; optional omission succeeds
STS-F03-07	AC-F03-07	Integration	Foreign option; duplicate question id; mixed text/option; 2,001-character text	POST each payload	`400 INVALID_ANSWER` for each
STS-F03-08	AC-F03-08	Integration (privacy)	resp1 has a response; authenticated as resp2	GET `/api/surveys/:id/responses/:responseId`	`403 FORBIDDEN_CROSS_RESPONSE_ACCESS`, no answer data
STS-F03-09	AC-F03-09	Integration (role)	Owner token; valid published survey	POST a response	`403 FORBIDDEN_ROLE`; no response stored
STS-F03-10	AC-F03-10	Integration	resp1 submitted	GET own receipt; reload and GET again	`200`, same response id, submitted time and answers

Concurrency note: STS-F03-04 sends two requests with `Promise.all` and asserts the multiset of statuses is `{201, 409}`. This is the test that must be shown red before the claim/unique guard and green after.

Part E - F-04 Aggregate Summary

5. Contract F-04: Aggregate Summary

5.1 Scope and linked requirements

- **Requirements:** FR-10
- **Business rules:** BR-04, BR-05, BR-11
- **NFRs:** NFR-SEC-02
- **Feature:** F-04
- **Dependencies:** F-01, F-02, F-03
- **Sprint:** 1

5.2 Behaviour

- Only the owner of a survey can view its summary.
- A respondent, or another owner, receives `403` with no summary data.
- The summary reports:
  - `totalResponses` - number of response rows for the survey.
  - Per question: `answeredCount`, option counts and percentages.
  - Short- and long-text answers as unlinked lists of strings.
  - Rating and integer count, average, minimum and maximum; rating also has a 1-5 distribution (Sprint 2).
  - Date answer counts grouped by calendar date (Sprint 2; F-06 Issue #9).
- Percentage denominator is `answeredCount` for that question, not `totalResponses`. This is correct when a question is optional and some respondents skip it.
- For multi-choice, each option percentage is the share of answerers selecting that option; totals may exceed 100% because one answer can select several options.
- Percentages are rounded to two decimals.
- No respondent name, email, user id or response id appears in the summary payload.
- A survey with zero responses returns zero counts and empty lists, not an error.

5.3 UI behaviour `/owner/surveys/:id/summary`

- Header: title, description, status badge, created date.
- KPI card: total responses.
- For each question:
  - Single-choice, multi-choice, yes/no and Likert: option label, progress bar, count and percentage.
  - Short and long text: list of unlinked answers, or "No text answers yet."
  - Rating (Sprint 2): count, average, observed minimum/maximum and distribution across 1-5.
  - Integer (Sprint 2): count, average, minimum and maximum.
  - Date (Sprint 2): count by calendar date.
- Empty state when `totalResponses = 0`: "No responses submitted yet."
- Loading skeleton and error banner.
- Access denied page for 403 without rendering any data.

5.4 Data specification Read-only aggregation over `survey_responses`, `answers`, `answer_options`, `options` and `questions`. F-06 supplies the typed answer fields; aggregation queries are scoped by `surveyId` and authorized by `ownerId`.

Percentage formula:

```
percentage = answeredCount > 0 ? round(optionCount / answeredCount *
    100, 2) : 0
```

5.5 API specification

GET /api/surveys/:id/summary Success 200:

```
{
  "success": true,
  "data": {
    "surveyId": "<uuid>",
    "title": "CPE 334 Midterm Feedback",
    "status": "PUBLISHED",
    "mode": "IDENTIFIED",
    "totalResponses": 15,
    "questions": [
      {
        "questionId": "<uuid-1>",
        "prompt": "How satisfied are you with the course?",
        "questionType": "SINGLE_CHOICE",
        "answeredCount": 15,
        "options": [
```

```
          { "optionId": "<uuid-opt-1>", "label": "Very satisfied", "
             count": 10, "percentage": 66.67 },
          { "optionId": "<uuid-opt-2>", "label": "Neutral", "count":
              4, "percentage": 26.67 },
          { "optionId": "<uuid-opt-3>", "label": "Unsatisfied", "
             count": 1, "percentage": 6.67 }
        ]
      },
      {
        "questionId": "<uuid-2>",
        "prompt": "What should improve?",
        "questionType": "SHORT_TEXT",
        "answeredCount": 8,
        "textAnswers": ["More lab time, please.", "Clearer grading
           rubric."]
      }
    ]
  }
}
```

Errors:

- `403 FORBIDDEN_ROLE` - respondent.
- `403 FORBIDDEN_OWNERSHIP` - another owner.
- `404 NOT_FOUND` - unknown survey.

5.6 Acceptance criteria

AC	Observable outcome
AC-F04-01	Counts, percentages and text answers match the database; percentages use `answeredCount` and are rounded to two decimals; no respondent identity field is present.
AC-F04-02	A respondent receives `403 FORBIDDEN_ROLE` and no summary data.
AC-F04-03	A non-owner owner receives `403 FORBIDDEN_OWNERSHIP` and no summary data.
AC-F04-04	A survey with zero responses returns `totalResponses = 0`, zero counts, empty text lists and `200`.

5.7 Software Test Specification File: `server/tests/survey-summary.test.ts`

Test ID	AC	Level	Precondition / input	Action	Expected result
STS-F04-01	AC-F04-01	Integration / DB	2 responses; option A chosen once, B once; one optional text answer	GET summary as owner	`200`, counts 1/1, percentages 50/50, `answeredCount` correct, text list present, no identity keys
STS-F04-02	AC-F04-02	Integration (role)	Authenticated respondent	GET summary	`403 FORBIDDEN_ROLE`, empty error data

Continued on next page

Test ID	AC	Level	Precondition / input	Action	Expected result
STS-F04-03	AC-F04-03	Integration (ownership)	Survey owned by A; authenticated as B	GET summary	`403 FORBIDDEN_OWNERSHIP`, no statistics
STS-F04-04	AC-F04-04	Integration	Published survey with no responses	GET summary	`200`, zero totals, empty option counts and text list

Part E (continued): Shared System Contracts and Test Specifications

6. Health Endpoint Reference

The scaffold's public `GET /api/health` contract is defined in Part C §4.4. Issue #2 verifies it; it is outside the feature-test count below and its latency is not evidence for NFR-PERF-01.

---

7. Supporting Unit Tests

These tests cover pure functions so important logic is verified without HTTP or database setup. They supplement, not replace, the integration tests.

File: `server/tests/unit/rules.test.ts`

Test ID	Target	Input	Expected
STS-U-01	Survey structure validator	Draft with zero questions	Invalid with `NO_QUESTIONS`
STS-U-02	Survey structure validator	Single-choice with one option	Invalid with `TOO_FEW_OPTIONS`
STS-U-03	Answer validator	Missing required question	Invalid with the missing id
STS-U-04	Answer validator	2,001-character text; empty required text	Invalid in both cases
STS-U-05	Percentage calculator	0 answered; 1/3 answered; 2/3 answered	`0`; `33.33`; `66.67`

---

8. Security Matrix Tests

Planned file: `server/tests/security-matrix.test.ts` (not present in the reviewed source snapshot).

Acceptance criteria:

AC	Observable outcome
AC-SEC-01	Every protected endpoint rejects a missing, malformed or expired token with `401` and returns no private data.
AC-SEC-02	Every forbidden role or ownership operation returns `403` with the correct code and leaves the database unchanged.

Test ID	Target	Method	Expected
STS-SEC-01	Every protected route (all 16 except login and health)	Call with no token, malformed token and expired token	`401` with `TOKEN_REQUIRED` or `INVALID_TOKEN`; no private data
STS-SEC-02	Role and ownership matrix from Part C §5.3	Call each operation with the wrong role and a non-owner	`403` with the correct code; database unchanged

The role matrix must include endpoints added after Sprint 1; the test enumerates the route table rather than a hand-written list that can drift.

---

9. End-to-End Tests

Planned file: `e2e/sprint1-survey-flow.spec.ts` (not present in the reviewed source snapshot).

Test ID	Journey	Expected
STS-E2E-01	Owner logs in, creates a survey, publishes it; respondent logs in, submits; owner views the summary	Survey published, response persisted, summary shows 1 response and 100% for the chosen option
STS-E2E-02	Respondent tries to submit twice, then opens another respondent's receipt URL, then logs out and reuses the old token	`409`, `403`, and `401` `TOKEN_REVOKED` respectively

E2E tests run against a clean seeded database and a production-like build.

---

10. Test Count Summary

Suite	File	Planned tests
F-01	`server/tests/auth.test.ts`	6
F-02	`server/tests/survey-lifecycle.test.ts`	11
F-03	`server/tests/response-submission.test.ts`	10
F-04	`server/tests/survey-summary.test.ts`	4
Security	`server/tests/security-matrix.test.ts`	2
Integration subtotal		**33**
Unit	`server/tests/unit/rules.test.ts`	5
E2E	`e2e/sprint1-survey-flow.spec.ts`	2
Total planned automated tests		**40**

This is the planned total. The dedicated security-matrix and E2E files were not present in the reviewed source snapshot; Part F records the implemented suites and reported results.

---

11. TDD Protocol for BR-01

BR-01 is the required TDD demonstration because it is a rule with a real concurrency risk.

Red stage

1. Confirm the frozen contract in `docs/sprints/sprint-1/f-03.md` and Issue #1. On the Issue #5 branch, commit the failing test before implementation.
2. Test: submit once (expect `201`), submit again (expect `409`), and submit two concurrent requests (expect one `201` and one `409`).
3. Run `npm test -- response-submission.test.ts`.
4. Record the real failure output. The expected failure is `409` received but `201` returned.
5. Commit message: `test(rules): add failing BR-01 duplicate submission test (red)`.

Green stage

1. Add the `ResponseClaim` check, unique constraint and `P2002` mapping.
2. Use a transaction so no partial response is stored.
3. Re-run the test until it passes.
4. Record the real passing output.
5. Commit message: `feat(rules): enforce BR-01 duplicate submission guard (green)`.

Refactor

- Extract the claim creation and error mapping into the response service.
- Keep the test unchanged; it must still pass.

Part F must include the two commit SHAs and the actual terminal excerpts. If the red stage cannot be reproduced, the team must say so rather than fabricate it.

---

12. Contract Version History

Version	Date	Change	Status
v1	4 Nov 2026	Original Thai draft; F-01..F-04 outlined; no edit/delete endpoints; single `selectedOptionId`; client-only logout; fabricated execution log	Superseded
v2	5 Nov 2026	English rewrite; added draft detail/update/delete endpoints; `AnswerOption` and `numberValue`; `ResponseClaim`; `tokenVersion`; corrected BR IDs; defined 40-test plan; removed fabricated evidence	Superseded
v3	5 Nov 2026	Aligned repository paths and issue links with the shared Kanban; included Docker Compose in Sprint 1; documented feature PRs to staging and release PRs to main	Superseded
v4	5 Nov 2026	Expanded the F-06 answer catalog; aligned rating range to 1-5 and documented text, integer and date value shapes and summaries	Superseded by v5
v5	6 Nov 2026	Moved the health endpoint contract to Part C; added shared unit and E2E test specifications; kept all F-06 acceptance criteria deferred under FR-12	Current baseline; implementation evidence is in Part F

The version history is kept so the change is visible. Contracts are not silently rewritten after implementation.

---

6 Part F: Sprint 1 Verification and Engineering Evidence

Evidence status: release promoted; Issue #7 worktree verification is complete, with manual and historical provenance items still open. Five Issue #7 screenshots were added by PR #16, from commit `b0e58eb9fbf066bc56ef5197ea5d39174f6afe76`. PR #17 was approved and merged from staging head `00494b26df89892066c2680e0a3e0cc03c905a7` to main as `827ed220e6d2290193df7d5234ed3b757ec38988`; tag `v0.1.0-sprint1` points to that commit. PR #17 reports typecheck, build, 56 tests, Compose image build and manual API checks passed on staging. Sprint 1 has no CI configured, so GitHub status checks are not expected; PR #17 is the recorded release verification and says source/test files match the release tree. The full browser flow was not rerun during release verification. The Issue #7 worktree based on `00494b26df89892066c2680e0a03e0cc03c905a7` ran on 13 November against a loopback-only PostgreSQL 18.1 `surveyhub_test`: `npm test` passed 7 suites / 60 tests; the focused security matrix passed 4/4 tests; `npm run test:e2e` passed both journeys; and 50 sequential reads measured p50 7.693 ms, p95 13.249 ms, p99 22.422 ms. These worktree results do not change the tagged release.

1. Version and reproduction summary

Repository: SurveyHub
Screenshot commit: `b0e58eb9fbf0` (PR #16, branch `feature/7-verify-release-deliverable-1`; screenshot-only change).
Prior application base: `2fda094fe986`.
Staging verification head: `00494b26df89`.
Release PR: #17, approved by YummieGG and merged to main on 13 November 2026 (Bangkok time).
Release tag: `v0.1.0-sprint1` points to merge commit `827ed220e6d2`.

The repository `README.md` contains full Docker and local setup instructions. The brief container path is:

```
cp .env.example .env
# Set local values in .env; do not commit secrets.
docker compose build app
docker compose up -d db
docker compose run --rm app npm run db:migrate:deploy
docker compose run --rm app npm run db:seed
docker compose up -d app
curl http://localhost:4000/api/health
```

2. Build and test results recorded by feature PRs

The results below are recorded in merged PR descriptions. PR #17 reports checks on staging head `00494b26df89`; it states that source and test files match the release tree. These are PR-recorded results, not independent reruns for this report.

Work	Result recorded in PR	Evidence
Scaffold (Issue #2)	Typecheck, build, health tests, Compose startup, health endpoint and restart-persistence checks reported passed	PR #10
F-01 Authentication	Targeted auth suite (9 tests) reported passed in PR #11; later full suite of 6 suites / 56 tests reported passed in PR #17	PRs #11 and #17
F-02 Survey lifecycle	32/32 tests across four suites, typecheck, build and repeatable seed reported passed	PR #12
F-03 Response rules	Five suites / 50 tests passed in PR #13; later full suite passed. PR #16 reports manual production-build checks for persistence, duplicate 409, cross-account 403 and revoked-token 401	PRs #13 and #16
F-04 Aggregate summary	Six suites / 54 tests reported passed in PR #14; later full suite passed in PR #17	PRs #14 and #17
Release verification	On staging head `00494b26df89`: typecheck, build, 6 suites / 56 tests, `docker compose build app`, manual API checks including receipt persistence after app-container recreation; browser login smoke passed. Full browser flow not rerun. Isolated `surveyhub_test` used	PR #17

PR #17 reports build, regression tests and manual staging API/persistence checks on head `00494b26df89`, and says source/test files match the release tree. Sprint 1 has no CI configured, so the lack of GitHub status checks is expected. The recorded staging verification is the release evidence; no separate run on the merge/tag commit is recorded. The Issue #7 worktree based on that staging head was rechecked on 13 November against an isolated loopback PostgreSQL 18.1 test database: `npm test` passed 7 suites / 60 tests; the focused security matrix passed 4/4 tests; `npm run test:e2e` passed 2/2 journeys; and the 50-request benchmark measured p50 7.693 ms, p95 13.249 ms, and p99 22.422 ms. Node was `v24.14.1` on Windows 10.0.26200 x64. Outputs are in `docs/evidence/npm-test-run.txt`, `docs/evidence/e2e-run.txt`, `docs/evidence/security-matrix-run.txt`, and `docs/evidence/nfr-perf-01-run.txt`. These are worktree results, not a separate run on the release tag.

3. Acceptance criteria to test results

`VERIFIED (PR report)` means a merged PR reports the mapped tests passing. PR #17 reports checks on staging head `00494b26df89` and says source/test files match the release tree. Sprint 1 has no CI configured; the PR's staging results are the available release verification, with no separate run on the merge commit recorded.

Acceptance criteria	Test source	Recorded result	Status
AC-F01-01–AC-F01-06	`server/tests/auth.test.ts`	9 targeted tests in PR #11; later full suite: 6 suites / 56 tests in PR #17	VERIFIED (PR report)
AC-F02-01–AC-F02-11	`server/tests/survey-lifecycle.test.ts`; `server/tests/unit/rules.test.ts`	32/32 tests reported passed in PR #12	VERIFIED (PR report)
AC-F03-01–AC-F03-10	`server/tests/response-submission.test.ts`	Full suite 6 / 56 passed per PR #17; PR #16 captures selected manual production-build checks	VERIFIED (PR report; selected manual flows)
AC-F04-01–AC-F04-04	`server/tests/survey-summary.test.ts`	54 targeted tests reported in PR #14; later full suite 6 / 56 passed per PR #17	VERIFIED (PR report)
AC-SEC-01–AC-SEC-02	`server/tests/security-matrix.test.ts`	All 4 matrix tests passed across the 14 protected route registrations; wrong-role and non-owner calls returned 403 without private data or database changes	PASS
AC-F01-06 / NFR-SEC-03	`server/tests/auth.test.ts`	Full suite passed the bcrypt cost-10 assertion and password-hash response privacy checks	PASS

Continued on next page

Acceptance criteria	Test source	Recorded result	Status
AC-PERF-01	`server/scripts/benchmark-read.ts`	50 sequential reads: p50 7.693 ms, p95 13.249 ms, p99 22.422 ms; p95 is below the 300 ms target	PASS
AC-DATA-01	Migration, submission tests and PR #17 manual staging API checks	Receipt and summary persisted after app-container recreation; database-container restart was not reported in release pass; tagged commit not separately rerun	PARTIAL (staging report)
STS-E2E-01–STS-E2E-02	`e2e/sprint1-survey-flow.spec.ts`; PRs #16 and #17	Production build and clean test-database setup completed; both Playwright journeys passed (2/2)	PASS

4. Demonstration scenarios

Five Issue #7 captures from PR #16 are embedded below. PR #17 separately reports manual staging API checks, including receipt persistence after app-container recreation. The full browser flow was not rerun during release verification, and the tagged merge commit has no separate test run.

Journey	Expected result	Evidence attached
Owner publishes a survey; respondent submits; owner opens summary	Persisted receipt and summary counts after app recreation	Figures 1 and 2
Respondent submits twice	Duplicate rejected with `409 DUPLICATE_SUBMISSION`	Figure 3; closed-survey rejection remains in source tests
Another respondent opens a receipt; user logs out and reuses the old token	`403 FORBIDDEN_CROSS_RESPONSE_ACCESS`; then `401 TOKEN_REVOKED`	Figures 4 and 5

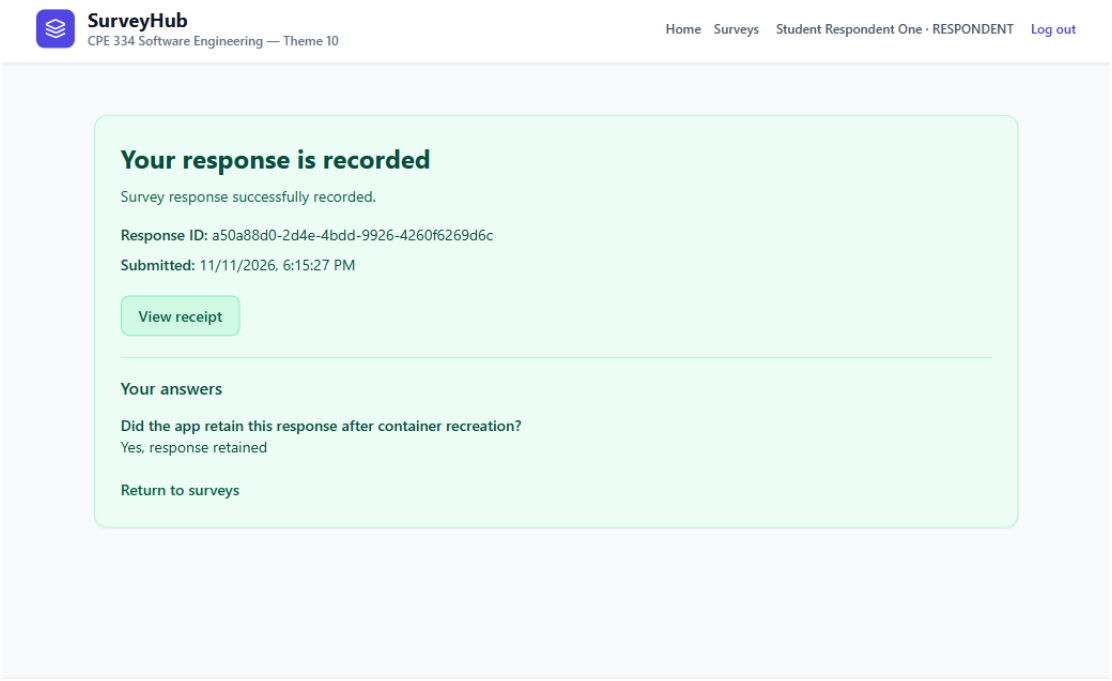


Figure 1: Respondent receipt and answer remain visible after app-container recreation.

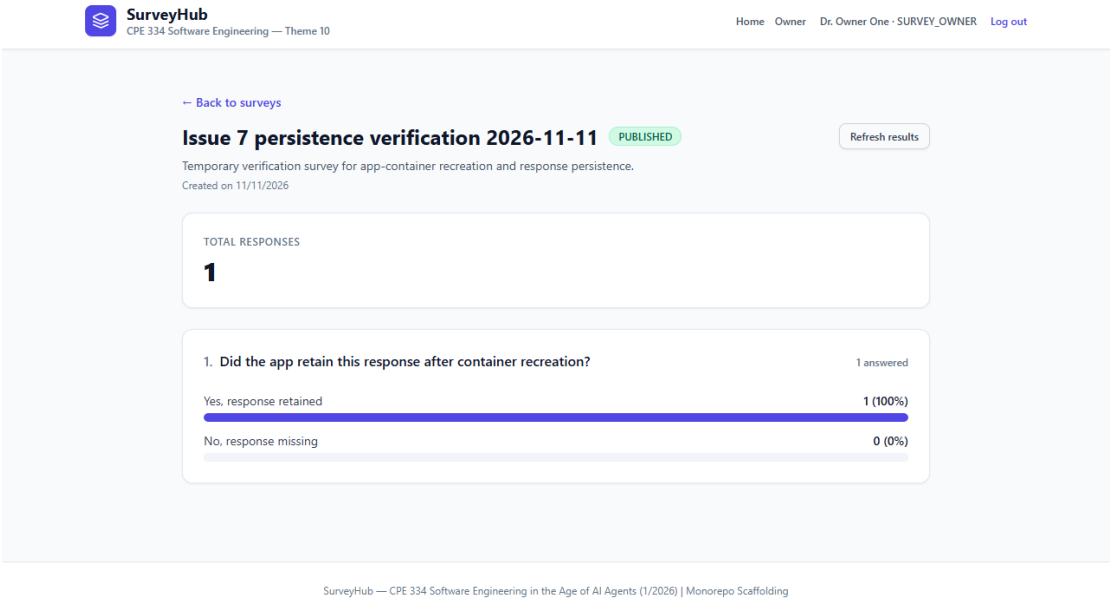


Figure 2: Owner summary after app-container recreation: one persisted response and its aggregate count.

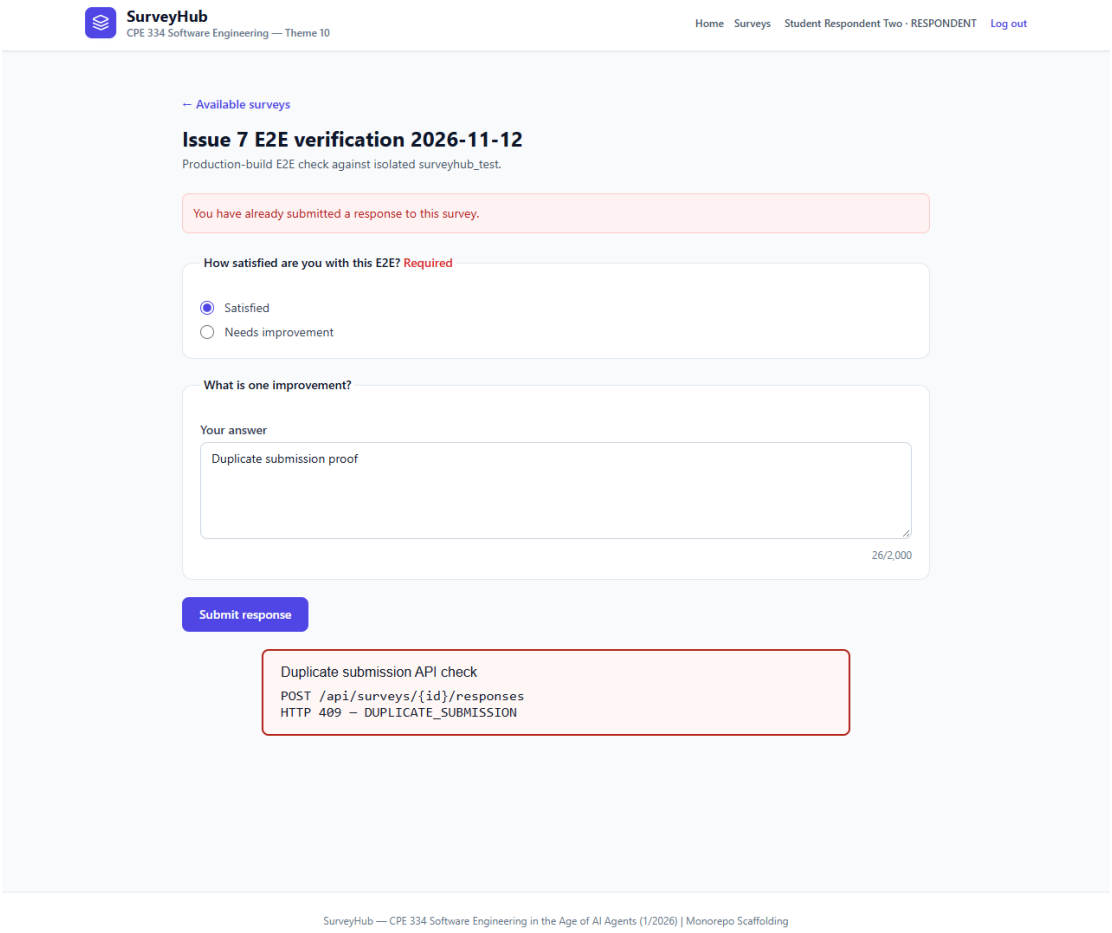


Figure 3: Duplicate submission is rejected with HTTP 409 `DUPLICATE_SUBMISSION`.

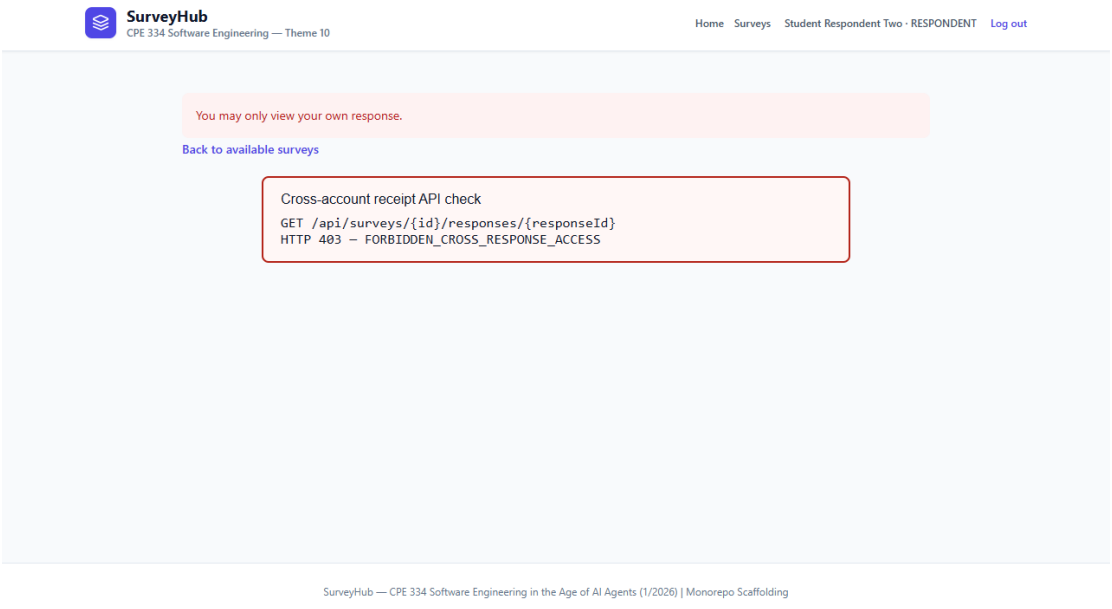


Figure 4: Cross-account receipt access is denied with HTTP 403 and no response data.

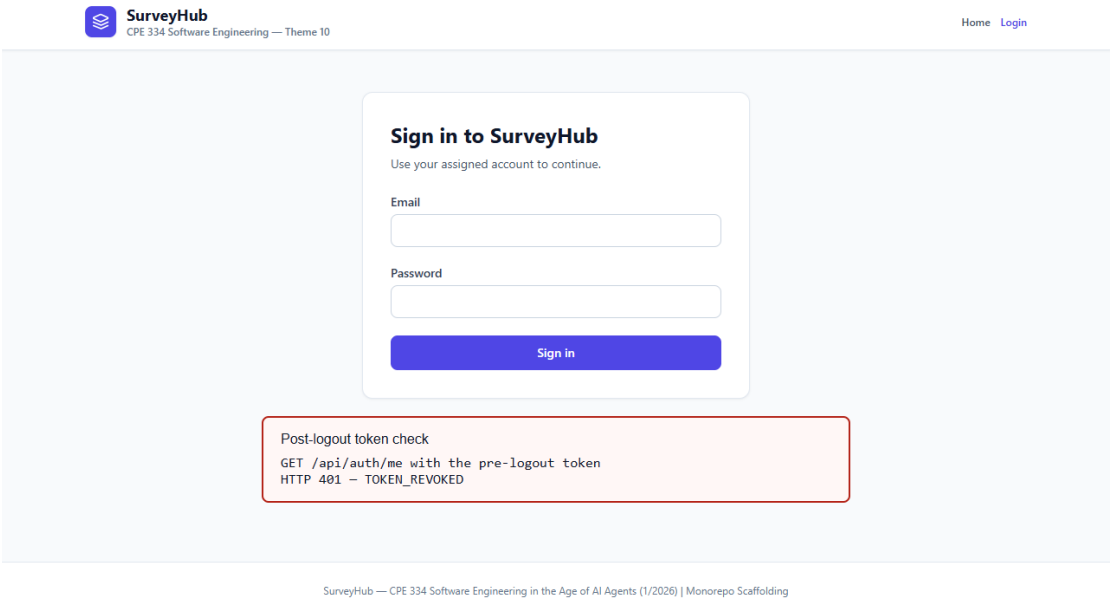


Figure 5: A pre-logout token is rejected with HTTP 401 `TOKEN_REVOKED`.

5. Actual test excerpts

These are excerpts from committed test source, not terminal output.

Duplicate and concurrent submission – response-submission.test.ts, lines 161–178:

```
expect(first.status).toBe(201);
expect(duplicate.status).toBe(409);
expect(duplicate.body.error.code).toBe('DUPLICATE_SUBMISSION');
expect(await prisma.surveyResponse.count({ where: { surveyId: survey
   .id, respondentId } })).toBe(1);
```

Respondent denied aggregate summary – survey-summary.test.ts, lines 159–168:

```
expect(res.status).toBe(403);
expect(res.body.error?.code).toBe('FORBIDDEN_ROLE');
expect(res.body.data).toBeUndefined();
```

6. TDD evidence for BR-01

Issue #5 TDD evidence records the corrected red/green sequence:

- **Red:** with the claim insert disabled and the duplicate indexes temporarily removed from the dedicated test database, `STS-F03-04` returned `201` instead of the expected `409`.
- **Green:** after restoring the claim transaction and both indexes, the same test passed with one `201`, one `409 DUPLICATE_SUBMISSION`, and one response/claim per respondent. The test database indexes were restored before the green run.
- An earlier red run failed with `404` because the submission route did not yet exist; the issue record says this was replaced because it did not isolate the duplicate guard.

The following terminal excerpts were recovered from the SEProject Codex chat for Issue #5. Both runs used the focused `STS-F03-04` test; the red setup disabled the claim insert and temporarily removed the unique indexes from the dedicated test database, while the green setup restored the claim transaction and indexes first.

Red run:

```
npm test --prefix server -- --runInBand --cacheDirectory .jest-cache
    --testTimeout=60000 tests/response-submission.test.ts -t "STS-
   F03-04"
FAIL tests/response-submission.test.ts (14.624 s)
  x STS-F03-04: returns one duplicate conflict sequentially and
     under concurrent submissions (1327 ms)
    Expected: 409
    Received: 201
Test Suites: 1 failed, 1 total
Tests:       1 failed, 9 skipped, 10 total
Ran all test suites matching /tests\\response-submission.test.ts/i
   with tests matching "STS-F03-04".
```

Green run:

```
npm test --prefix server -- --runInBand --cacheDirectory .jest-cache
    --testTimeout=60000 tests/response-submission.test.ts -t "STS-
   F03-04"
PASS tests/response-submission.test.ts (16.355 s)✓
   STS-F03-04: returns one duplicate conflict sequentially and under
       concurrent submissions (1676 ms)
Test Suites: 1 passed, 1 total
Tests:       9 skipped, 1 passed, 10 total
Ran all test suites matching /tests\\response-submission.test.ts/i
   with tests matching "STS-F03-04".
```

These are terminal-output excerpts captured in the project chat, not a separately archived raw log. Issue #5 comments and the reachable Git history do not associate either run with a commit SHA, so the exact red and green SHAs remain unavailable and are not inferred from nearby feature-branch commits. The corrected red/green summaries are preserved as reported evidence; no new controlled TDD red/green reproduction was run, and the passing regression suite does not recreate the historical database changes used for the red stage.

7. Selected issue, PR, review and merge chains

Issue / PR	Branch; author / reviewer	Review history	Merge / release	Evidence
#5 / #13	`feature/5-one-response-per-survey`; Snnn3 / YummieGG	Changes requested; approved after validation, accessibility and UI fixes	10 Nov 2026; merge `8b609073`	PR checks passed; no manual browser test
#6 / #14	`feature/6-show-aggregate-results`; YummieGG / Snnn3	Three change requests; approved after authorization, race and refresh fixes	10 Nov 2026; merge `2fda094`	Final reviewer did not rerun tests, build or CI
#7 / #16	`feature/7-verify-release-deliverable-1`; Snnn3 / YummieGG	Approved; screenshot-only change	12 Nov 2026; staging merge `00494b26`	Five screenshots; manual checks
PR #17 release	`sprint-1-staging`; Snnn3 / YummieGG	Approved; merged to main	13 Nov 2026; merge/tag `827ed220`	Tag `v0.1.0-sprint1`; staging verification recorded; Sprint 1 has no CI

8. Contribution record

Workstream	PR author	Evidence
Parts A–E baseline	Snnn3	PR #8
Scaffold, schema and seed	YummieGG	PR #10
F-01 Authentication	Snnn3	PR #11
F-02 Survey lifecycle	YummieGG	PR #12
F-03 Response rules	Snnn3	PR #13; reviewed by YummieGG

Continued on next page

Workstream	PR author	Evidence
F-04 Aggregate summary	YummieGG	PR #14; reviewed by Snnn3
Screenshot and release evidence	Snnn3	PR #16; approved by YummieGG
Staging-to-main release	Snnn3	PR #17; approved by YummieGG

This is a PR-authorship summary, not an effort-percentage estimate. Work done outside these PRs and the review attribution for other workstreams are not established by the records cited here.

9. Sprint 1 evidence gaps, failures and disclosures

Item	Status
NFR-PERF-01	50 sequential requests measured p50 7.693 ms, p95 13.249 ms, p99 22.422 ms against the 300 ms p95 target; see `docs/evidence/nfr-perf-01.json`
E2E automation and run output	Both Playwright journeys passed against a clean seeded test database after the production build; see `docs/evidence/e2e-run.txt`
Security matrix	All 4 focused matrix tests passed. The 14 protected routes returned the required 401/403 responses without private data or database changes; the bcrypt cost-10 assertion also passed in the full suite
Versioned verification record	The Issue #7 worktree records passing E2E/security/performance results and evidence paths; remaining manual scenarios and release-tag verification are pending
Release evidence	PR #17 approved/merged; `v0.1.0-sprint1` points to `827ed220e6d2`. PR #17 reports checks on staging head `00494b26df89` and says source/test files match the release tree. Sprint 1 has no CI; no separate post-merge run is recorded
TDD artifacts	Red/green terminal-output excerpts are recovered from the Issue #5 Codex project chat and included above; a separately archived raw log and exact per-stage commit SHAs are unavailable

PR #13's transient Prisma DLL `EPERM` passed on sequential rerun; PR #17 reports 56 tests passed on its staging head. PRs #16/#17 cover selected flows only. The Issue #7 worktree now records typecheck/build, 7 suites / 60 tests, 2/2 E2E journeys, the 4/4 security matrix, and the measured 50-request benchmark. Manual scenarios and separate release-tag verification remain pending.

AI-use disclosure: Codex chats document Sprint 1 assistance with Issues #3 (F-01) and #5 (F-03), test/debug work, review of PR #10, and this report's LaTeX. Each student must confirm their own AI use and review; these chats do not establish use by other members.

Evidence still needed: the test and benchmark runs above are worktree results, not a separate run on the release tag. Manual scenarios not covered by saved screenshots and the students' contribution, review, and AI-use statements remain pending. Historical TDD stage SHAs remain unavailable after Git and Issue #5 searches; the excerpts are chat-recovered, not archived raw logs.